

PROJEKAT USMJERENJA EMBEDED SISTEMI I ALGORITMI

Prebrojavanje ljudi prebrojavanjem lica

Kratak opis:

U ovom fajlu nalazi se kompletna dokumentacija za projekat usmjerenja Embedded sistemi i algoritmi, koja pokriva četiri predmeta koji se polažu u sklopu projekta. Svaki predmet predstavlja posebnu sekciju ovog dokumenta i jasno je naznačen.

Sandić Vojislav, Glišević Sara, Jović Radivoje

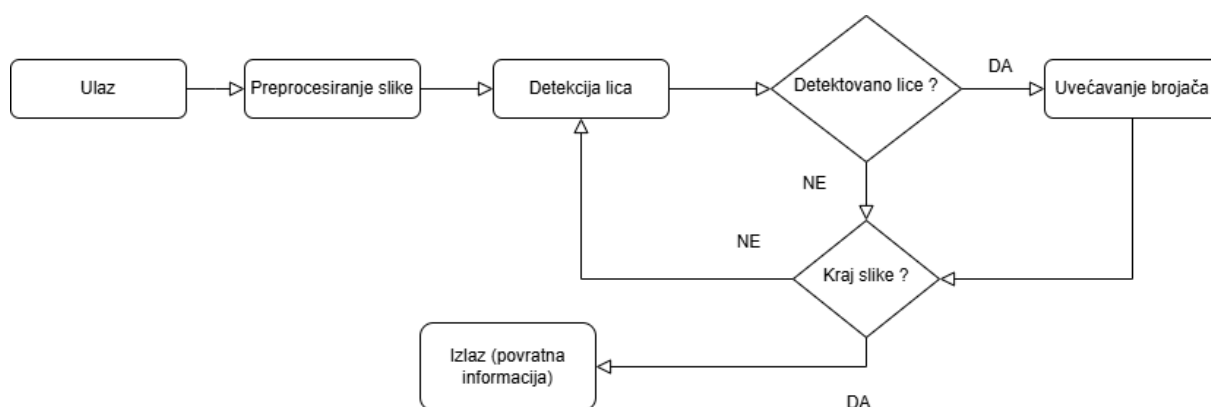
SADRŽAJ:

PROJEKTOVANJE ELEKTRONSKIH UREĐAJA NA SISTEMSKOM NIVOU	2
1. Ideja projekta	2
1.1 Ograničenja sistema	3
2. Matematički model sistema.....	4
3. Izvršna specifikacija	8
3.1 main.cpp	10
3.2 facedetect.cpp	12
4. Analize prije partitionisanja.....	16
4.1 Vremenska analiza	16
4.2 Algoritamska analiza.....	18
4.3 Bitska analiza.....	20
5. Partitionisanje.....	24
5.1 Virtuelna platforma	24
5.2 Registarske mape	32
6. Analize poslije partitionisanja.....	34
6.1 Analiza performansi.....	34
6.2 Analiza površine	38
6.3 Analiza potrošnje i interfejsa.....	39
7. Reference	41

PROJEKTOVANJE ELEKTRONSKIH UREĐAJA NA SISTEMSKOM NIVOU

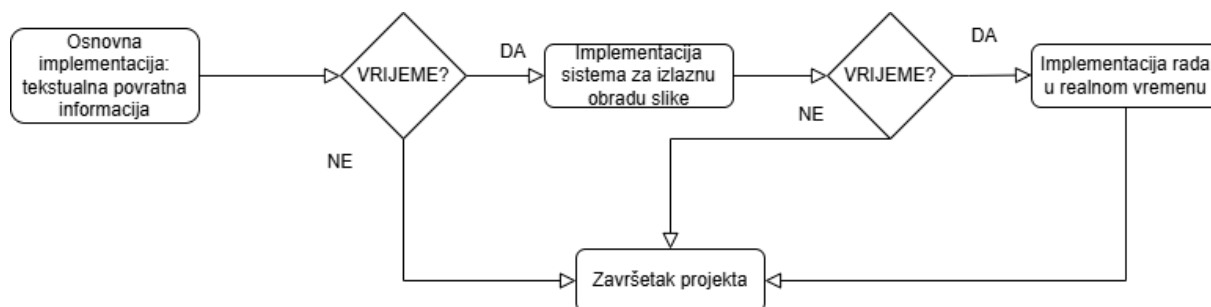
1. Ideja projekta

Detekcija lica predstavlja problem binarne klasifikacije kombinovan sa problemom lokalizacije : na osnovu fotografije, treba odlučiti da li ona sadrži lica, te ista jasno ograničiti na toj fotografiji. Ideja našeg projekta jeste da na osnovu ulaza u sistem (fotografije ili frejma sa kamere) sistem detektuje lica na ulaznom frejmu i na izlazu vraća povratnu informaciju o tome koliko lica isti frejm sadrži. Prosta blok šema koja opisuje rad sistema, na najvišem nivou apstrakcije, prikazana je na slici 1:



Slika 1. Opis rada sistema za obradu slike na najvišem nivou apstrakcije

Konkretno, obrada slike u ovom slučaju podrazumijeva da na odgovarajućem ulazu na određeni način prilagodimo sliku svojim potrebama i na istoj prepoznamo lica, koja će odgovarajuća softverska brojačka logika pri detekciji brojati dok se obrada slike ne završi. Ovi procesi se izvršavaju skoro paralelno, ali je radi jednostavnosti blok dijagrama, a i veće važnosti za ispravan rad sistema, prednost data procesu obrade slike. Razvojni put kojim će se ovaj tim voditi pri realizaciji projekta dat je na slici 2.



Slika 2. Razvojni put (roadmap) projekta

Osnovna faza projekta podrazumijeva implementaciju logike za detekciju i prebrojavanje lica na slici, te povratnu informaciju kojom sistem obavještava korisnika o broju detektovanih lica. Zavisno od

preostalog vremena nakon završetka osnovne faze, moguće su dodatne implementacije u vidu izlazne obrade slike, koja će pored ispisa povratne informacije osigurati da izlaz bude i obrađena slika sa jasno označenim regionima slike u kojima su detektovana lica. Posljednji dodatni korak podrazumijeva konfiguraciju sistema tako da bude sposoban za rad u realnom vremenu, za šta je potrebno određeno dodatno planiranje zavisno od raspoloživih resursa.

Konkretni cilj kao ishod projekta jeste sistem sposoban za prepoznavanje lica koji će moći da se koristi u situacijama gdje je osvjetljenje prihvatljivo i koji će biti sposoban za rad sa ograničenim resursima. Primjeri potencijalne primjene ovog sistema su razni sistemi za video-nadzor, npr. takozvane kamere za zvona (engl. **doorbell** camera), koje bi vlasnicima slale informacije u slučaju da primjete željene/neželjene posjetioce, zatim sistemi za nadzor bankomata, javnih ustanova, automatska vrata i slično.

Daljim doradama ovog sistema moguće je napraviti sisteme kao što su biometrijske brave, face identification sistemi i drugo.

Minimalan izlaz koji očekujemo kao rezultat rada uređaja prikazan je na sljedećem kodnom listingu:

```
-- ulazak u main funkciju --
-- učitavanje slike --
-- slanje podataka --
-- detektovanje lica --
tb: face_num_out=8
velicina rezultata: 8
```

Terminalski ispis 1. Minimalni očekivani izlaz kao rezultat rada sistema.

1.1 Ograničenja sistema

Uređaj na kome se nalazi aplikacija je Zybo Z7-10 razvojna ploča. Centralna komponenta ploče je XC7Z010 SoC koji kombinuje ARM Cortex-A9 procesor i FPGA dio čipa. Složenost implementiranog dizajna u FPGA logici je ograničena brojem standardnih ćelija, što se vidi u tabeli ispod. Ovim je ograničena hardverska akceleracija. RAM memorija je DD3L memorija veličine 1GB sa brzinom do 533 MHz i 32-bitnom magistralom. Pored nje tu je i 16 MB Quad-SPI Flash memorija. Procesor posjeduje 8 I/O pinova. FPGA deo posjeduje 32 I/O pina. Uređaj se napaja pomoću USB porta ili bilo kojeg drugog spoljašnjeg izvora napajanja od 5V. Uređaj za hlađenje ne postoji.

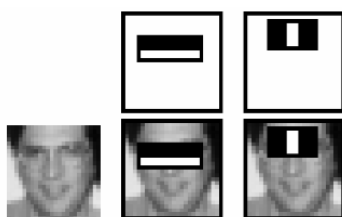
Broj LUT-ova	17600
Broj flip-flopova	35200
Blok RAM (BRAM)	270 KB

Tabela 1. FPGA resursi

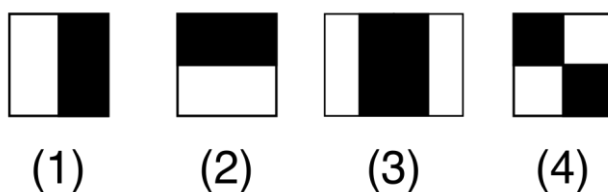
Što se tiče softverskih ograničenja, ARM dual core Cortex-A9 je dvojezgarni procesor koji radi na maksimalnoj frekvenciji od 667 MHz. Može da podrži operativni sistem poput PetaLinux-a. Zbog ograničenja ploče nije praktično pokretati operativne sisteme poput Ubuntu-a.

2. Matematički model sistema

Što se tiče same matematičke pozadine koda, potrebno je osvrnuti se na algoritam koji je dio specifikacije. Viola-Jones algoritam baziran je na **Haarovim karakteristikama** digitalne slike. Haarove karakteristike prvi su definisali **Paul Viola** i **Michael Jones**, po kojima je algoritam i dobio ime, kako bi riješili problem računske skupoće rada sa intenzitetima slike. Haarove karakteristike su digitalne karakteristike slike koje se koriste za prepoznavanje objekata. Ime duguju svojoj intuitivnoj sličnosti sa Haarovim talasima. Ukratko, Haarova karakteristika posmatra susjedne pravougaone regione **greyscale** slike na specifičnim lokacijama u detekcijskom prozoru, sumira intenzitete piksela u svakom regionu i računa razliku između ovih suma. Razlika se zatim koristi za kategorizaciju dijelova slike. Ilustracija načina rada, kao i rezultat izvršavanja specifikacije, dat je na sljedećim slikama:



Slika 3. Način izvlačenja Haarovih karakteristika



Slika 4. Haarove karakteristike slike po Violi i Jonesu

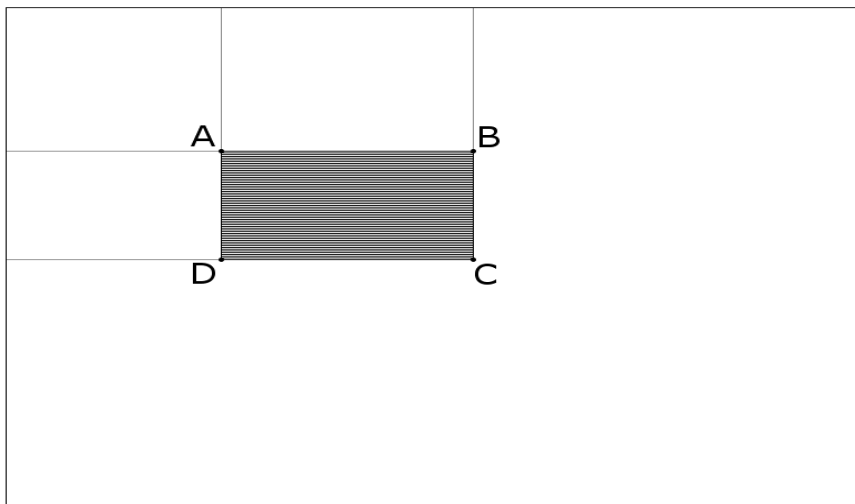
U fazi detekcije, prozor određene veličine se prevlači preko ulazne slike, pa se za svaku podsekciju slike računaju Haarove karakteristike. Ova razlika se zatim poredi sa prethodno naučenim vrijednostima koje razdvajaju objekte (lica u našem slučaju) od onih koji to nisu. Pošto je jedna takva Haarova karakteristika sama po sebi **weak learner** (malo bolja od nasumičnog pogađanja), potreban je veliki broj takvih karakteristika da bi se objekat dovoljno precizno opisao. Zbog toga, u Viola-Jones algoritmu se te osobine kaskadno organizuju tako da postanu jak klasifikator (**strong learner**). Ovakav način je računski brz zbog upotrebe **integralnih slika**, i može vršiti proračune konstantnom brzinom (60 instrukcija za jednu 2-pravougaonu osobinu).

Integralne slike (**summed-area tables**) su zapravo dvodimenzionalni LUT-ovi u formi matrice iste veličine kao i originalna. Integralna slika je način predstavljanja slike gdje svaki piksel sadrži zbir svih piksela iz originalne slike koji se nalaze **iznad i lijevo** od njega (uključujući i taj piksel).

Ovo omogućava brzo izračunavanje zbira piksela unutar bilo kojeg pravougaonog regiona slike.

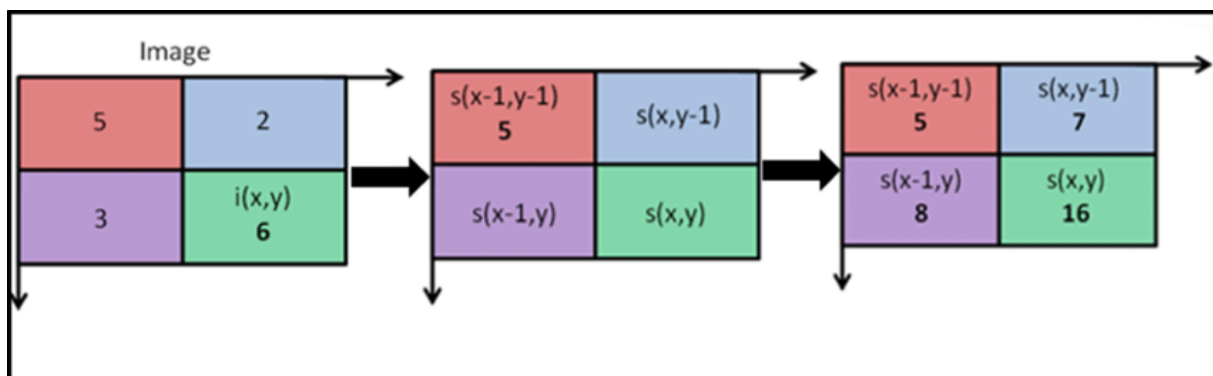
$$(2.1) \quad S = I(C) + I(A) - I(B) - I(D)$$

gdje tačke A,B,C,D pripadaju integralnoj slici, kao što je prikazano na slici ispod:



Slika 5. Nalaženje sume označene pravougaone zone

Primjer računanja integralne slike dat je na slikama 7 i 8 :



Slika 6. Ilustracija računanja integralnih slika

Integral Image

Input Image:

1	5	2
2	4	1
2	1	1

Integral Image:

1	6	8
3	12	15
5	15	19

Slika 7. Ilustracija računanja integralnih slika

Učenje ovog algoritma vrši se tako što se od svih Haarovih osobina u određenoj oblasti ($M = 162336$ za preporučenu veličinu oblasti potprozora od 24×24 piksela) vrši odabiranje najbitnijih pri treniranju. To odabiranje se vrši upotrebom **AdaBoost** pomoćnog algoritma. **AdaBoost** (skraćeno od **Adaptive Boosting**) je meta-algoritam za statističku klasifikaciju koji su razvili **Yoav Freund** i **Robert Schapire** 1995. godine, za šta su 2003. dobili Gödelovu nagradu. Može se koristiti u kombinaciji sa različitim tipovima algoritama za učenje radi poboljšanja performansi. Izlazi više slabih klasifikatora kombinuju se u ponderisanu sumu koja predstavlja konačni rezultat ojačanog klasifikatora. Obično se AdaBoost koristi za binarnu klasifikaciju, iako se može generalizovati na više klasa ili ograničene intervale realnih vrednosti. Algoritam funkcioniše po sljedećoj formuli :

$$(2.2) \quad h(x) = \text{sgn}\left(\sum_{j=1}^M \alpha_j h_j(x)\right)$$

Gdje su h_j slabi klasifikatori koji se definišu na sljedeći način :

$$(2.3) \quad h_j(x) = \begin{cases} -s_j, & f_j < \theta_j \\ s_j, & \text{drugačije} \end{cases}$$

θ_j - granična vrijednost, s_j (može biti ± 1) - polaritet , kao i alfa koeficijenti se definišu tokom treniranja modela. Rezultat ovakvog odabiranja jesu jači klasifikatori, a broj osobina se znatno smanjuje (na oko 6000). Kako bi se poboljšala efikasnost, takvi jaki klasifikatori se međusobno kaskadiraju, te zavisno od broja faza, ukoliko region slike ne prođe provjeru na prvom klasifikatoru, pri poređenju sljedeći klasifikator taj region slike uopšte neće uzimati u obzir.

Ukratko, Viola–Jones je u suštini boostovani algoritam za učenje karakteristika, treniran primjenom modifikovanog AdaBoost algoritma na klasifikatore sa Haarovim karakteristikama radi pronalaženja sekvenci klasifikatora. Haar klasifikatori karakteristika su grubi, ali omogućavaju veoma brzo izračunavanje, dok modifikovani AdaBoost izgrađuje jak klasifikator od mnoštva slabih klasifikatora.

Tokom rada, slika se testira na svakom klasifikatoru uzastopno. Ukoliko u bilo kom trenutku određeni klasifikator vrati 0, algoritam odmah daje rezultat „nema detektovanog lica“. Ako svi klasifikatori vrate 1, algoritam daje rezultat „lice detektovano“. Zbog toga se Viola–Jones klasifikator naziva i „Haar kaskadni klasifikator“.

3. Izvršna specifikacija

U ovom odjeljku biće data izvršna specifikacija sistema te će biti dato objašnjenje bitnih dijelova iste i bitnih odluka potrebnih u dizajnu navedenog uređaja.

Specifikacija za navedeni sistem pisana je u programskom jeziku MATLAB, zbog toga što je MATLAB jezik visokog nivoa sa određenim ugrađenim funkcijama koje olakšavaju pisanje specifikacije, iako je cijena toga što se pisanjem specifikacije upotrebom ugrađenih funkcija sistem ne apstrahuje previše.

Specifikacija je napisana pod pretpostavkom da je u pitanju sistem sa ograničenim resursima, konkretno FPGA sistem, pa je cilj da kod bude optimalan za rad na jednom takvom sistemu.

U priloženoj specifikaciji prvo se vrši inicijalizacija promjenljive koja predstavlja sliku koju obrađujemo. U slučaju da je ulaz sistema video sa kamere u realnom vremenu, zapis je nešto složeniji, pa ćemo pretpostaviti da je na ulazu sistema slika, a odgovarajuće adaptacije praviti u daljem razvoju i pisanju dokumentacije. Prvo je potrebno izvršiti provjeru da li je u odgovarajućem folderu postoji slika koju tražimo korištenjem **isfile** funkcije. Nakon toga, upotrebom ugrađene funkcije **imread** vrši se čitanje slike sa zadate putanje. Poslije toga vrši se sama detekcija i prebrojavanje lica, o kojima će više riječi biti u narednim pasusima, jer tu dolazi do prethodno pomenutog smanjenja apstrakcije u samoj specifikaciji. U implementaciji koja sadrži izlaznu obradu slike, potrebno je označiti lica na slici, što je u MATLABU omogućeno korištenjem funkcije **insertObjectAnnotation**, koja u sliku ubacuje granične pravougaonike oko regiona u kojima su detektovana lica. Funkcijom **disp** se zatim ispisuje broj lica koja su pronađena na slici, što emulira ispis u terminalu koji očekujemo kao izlaz. Dodatno, za implementaciju sa izlaznom obradom slike, potrebno je sliku sa ubačenim graničnim pravougaonicima sačuvati kao izlaz, što se postiže upotrebom **imwrite** funkcije i slanjem promjenljive koja predstavlja obrađenu sliku i izlazne putanje kao parametara.

Najzanimljiviji kodni listing tiče se detekcije ili prebrojavanja lica. Razlog leži u tome što su za implementaciju tih funkcionalnosti korištene ugrađene funkcije MATLABA koje predstavljaju dijelove **Viola-Jones** algoritma za detekciju lica. Kod koji implementira navedeni algoritam, o kome će detaljnije biti diskutovano u nastavku, jeste:

```
detektorLica =vision.CascadeObjectDetector('haarcascade_frontalface_default.xml');  
bbox = step(detektorLica, img);  
brojLica = size(bbox, 1);
```

Listing 1. Implementacija detekcije i prebrojavanja lica

Ideja iza ovog koda jeste sljedeća:

```
detektorLica =vision.CascadeObjectDetector('haarcascade_frontalface_default.xml');
```

-napravi objekat **detektorLica** koji sadrži parametre potrebne za detekciju lica prethodno pomenutim algoritmom, a koje preuzima iz odgovarajućeg XML fajla koji predstavlja prethodno trenirani model za taj algoritam

```
bbox = step(detektorLica, img);
```

-funkcijom **step** se detektuju granični regioni na slici upotrebom prethodno kreiranog objekta

```
brojLica = size(bbox, 1);
```

-ova linija koda vrši prebrojavanje lica tako što provjeri veličinu matrice - rezultata step funkcije koja predstavlja niz koordinata na kojima su detektovana lica. Veličina te matrice ujedno predstavlja i broj lica koja su prebrojana na slici. Rezultat izvršavanja specifikacije nalazi se u listingu 2, dok se na slici 3 nalazi očekivani rezultat izlazne obrade slike.

```
>> specifikacija
```

Broj prebrojanih lica: 11

Rezultati sačuvani u: oznacena_slika.jpg

Listing 2. Ispis koji očekujemo kao izlaz specifikacije, emuliran u MATLAB-u



Slika 8. Izlazno obrađena slika

Kako bismo nastavili sa daljim analizama, specifikaciju prevodimo u jezik nižeg nivoa, što će u našem slučaju biti C++ programski jezik. Izvršna specifikacija time postaje C++ kod. Kao softverski model, izvršna specifikacija je **sekvencijalna na osi konkurentnosti ESL taksonomije**. Pristup izvornim fajlovima je moguć čineći sistem dizajnom (eng. Design) na osi konfigurabilnosti. Prevedena specifikacija se sastoji iz sljedećih fajlova:

define.h – sadrži definicije konstanti i tipova potrebnih za rad algoritma.

image.cpp/image.h – sadrže definicije i implementacije funkcija potrebnih za rad sa greyscale slikama.

image_ops.cpp/image_ops.h – sadrži definicije konstanti i tipova potrebnih za preprocessing operacije nad .jpg slikom kako bi bila pogodna za algoritam (resizing, greyscaling).

stb_image.h/stb_image_resize.h/stb_image_write.h – pomoćne biblioteke potrebne za preprocessing operacije nad slikom.

main.cpp – predstavlja glavnu funkciju u specifikaciji, uprošćena tako da samo instancionira objekat klase **test_FACEDETECT** i poziva njegovu glavnu metodu, u kojoj se dalje sve izvršava.

tb_facedetect.cpp/tb_facedetect.h – sadrže definiciju implementaciju čitavog workflowa algoritma, povezuju preprocessing naredbe sa naredbama detekcije lica i vraćaju broj detektovanih lica kao rezultat svog izvršavanja.

facedetect.cpp/facedetect.h – sadrži dio od interesa za rad samog algoritma (metode koje vrše sami proces detekcije lica).

3.1 main.cpp

U ovom odjeljku ćemo posebno analizirati **main.cpp**, odnosno **tb_facedetect.cpp** koji main funkcija poziva. Kodni listing za main funkciju dat je u nastavku :

```
#include "define.h"

#include "facedetect.h"
#include "tb_facedetect.h"

// top funkcija sistema
int main(int argc, char** argv)
{
    // kreiranje testbencha i pokretanje
    test_FACEDETECT test;
    test.test_main();

    return 0;
}
```

Listing 3. Kodni listing main fajla

Kao što smo već ustanovili, main funkcija isključivo služi kako bi instancionirala objekat klase **test_FACEDETECT**, koji zapravo povezuje sve komponente. Listing njegove glavne metode, **test_main**, je takođe dat u nastavku:

```
void test_FACEDETECT::test_main()
{
    int flag;
    int face_number;
    int shiftStep;
    float scaleFactor;
    vector<MyRect> result;
    FILE *fp;

    printf("-- priprema slike za main funkciju --\r\n");

    // preprocessing: konverzija ulaznog jpg u greyscale pgm
    operation();

    printf("-- ulazak u main funkciju --\r\n");
    printf("-- učitavanje slike --\r\n");

    flag = readPgm((char *)"Face.pgm", image); // citanje .pgm fajla
```

```

if (flag == -1)
{
    printf("Ne mogu otvoriti .pgm fajl\n");
    return;
}

printf("-- slanje podataka --\r\n");

// citanje iz parameter.txt fajla
fp = fopen("../data/parameter.txt","r");
if (!fp){
    printf("Nemoguće citanje iz parameter.txt\n");
    return;
}
fscanf(fp,"%f",&scaleFactor); // prva linija u parameter.txt
fscanf(fp,"%d",&shiftStep);    // druga linija u parameter.txt
fclose(fp);

// kopiranje podataka iz image strukture u 2D bafer koji očekuje
facedetect
uint8_t img_buf[IMAGE_HEIGHT][IMAGE_WIDTH];
for(int i = 0; i < IMAGE_HEIGHT; i++)
    for(int j = 0; j < IMAGE_WIDTH; j++)
        img_buf[i][j] = image->data[i * IMAGE_WIDTH + j];

printf("-- detektovanje lica --\r\n");

// kreiranje i pokretanje detektora
facedetect u_FACEDETECT;
u_FACEDETECT.detection_main(img_buf, scaleFactor, (uint8_t)shiftStep);

face_number = u_FACEDETECT.face_num_out;
printf("tb: face_num_out=%d\n", face_number);

printf("velicina rezultata: %d\n", (int)result.size());

// brisanje pomocne slike
remove("Face.pgm");

/* brisanje slike i oslobadjanje klasifikatora */
freeImage(image);
}

```

Listing 5. Kodni listing test_main funkcije

Kao što se može i primjetiti, ova funkcija poziva sve ključne korake za rad izvršne specifikacije, vrši obradu ulazne slike, pretvarajući sliku .jpg formata u 8-bitnu greyscaled sliku fiksne rezolucije 360 x 240, koja predstavlja ulaz samog algoritma. Zatim učitava ulazne podatke potrebne za izgradnju piramide integralnih slika (scaleFactor i shiftStep), kopira sliku u ulazni bafer te započinje proces detekcije. Nakon završene detekcije funkcija vraća broj detektovanih lica, oslobađa bafere i briše pomoćni fajl za greyscale sliku. Time je proces detekcije završen.

3.2 facedetect.cpp

U datoteci facedetect.cpp implementiran je čitav proces detekcije lica. U nastavku će biti data objašnjenja pojedinih funkcija, pa ćemo na osnovu njih zaključiti koje od tih funkcija su pogodne za hardversku akceleraciju, što će dalje biti diskutovano u odjeljku Vremenska analiza. Funkcije o kojima je potrebno prodiskutovati nalaze se u zaglavlju **facedetect.h** i one su redom :

```
/*priprema slike za Haarovu kaskadnu klasifikaciju*/
void setImageForCascadeClassifier(int* sum, int* sqsum, int width);

/*azuriranje vrijednosti piksela*/
void updatePvalue(int* sum, int* sqsum, int p_offset, int pq_offset,
int width);

/*Pokretanje kaskade na odredjenom prozoru*/
int runCascadeClassifier(MyPoint pt, int start_stage, int width);

/*grupisanje pravougaonika koji se preklapaju pri detekciji */
void groupRectangles(int groupThreshold, float eps);

/*grupisanje pravougaonika u klastere na osnovu preklapanja */
int partition(int* labels, float eps);

/*poredjenje pravougaonika bazirano na blizini radi grupisanja u
klastere */
int predicate(float eps, uint16_t r1[4], uint16_t r2[4]);

//progresivno skaliranje slike na 24*24 prozor radi detekcije lica
razlicitih velicina
void ScaleImage_Invoker(float factor, int sum_col, int shift_step, int
y_bias);

//evaluacija slabih klasifikatora, rad Haarovog filtra
int evalWeakClassifier(int variance_norm_factor, int p_offset, int
tree_index, int w_index, int r_index);

//funkcija za rad sa integralnim slikama
void integralImages(uint8_t src[IMAGE_HEIGHT][IMAGE_WIDTH], int
*sumData, int *sqsumData, int width, int height);

//posebna funkcija za rad s integralnim slikama u posljednjem redu
void integralimages_lastrow(uint8_t src[IMAGE_HEIGHT][IMAGE_WIDTH], int
*sumData, int *sqsumData, int width, int y_bias);

//nearest neighbor algoritam, koristi se za downsampling slike kako bi
se izgradila piramida slika
void nearestNeighbor(uint8_t dst[IMAGE_HEIGHT][IMAGE_WIDTH], int width,
int height);

//funkcija koja poziva sve glavne korake
void detectObjects(MySize minSize, float scale_factor, int
min_neighbors, int shift_step);

//main funkcija
void detection_main(uint8_t image_data[IMAGE_HEIGHT][IMAGE_WIDTH],
float scaleFactor_val, uint8_t shiftStep_val);
```

Listing 6. Kodni listing zaglavlja funkcija u facedetect.h fajlu

Glavna funkcija ovdje je `detection_main`, koja prima `scaleFactor` i `shiftStep` definisane u `parameter.txt` fajlu kao ulaze, zajedno sa podacima o slici koja se iščitava. Njena uloga je zapravo da konfiguriše neophodne parametre algoritma (veličine prozora, broj susjednih pravougaonika,...), dok se detekcija lica zapravo odigrava u funkciji `detectObjects`. Njen listing prikazan je u nastavku:

```
void facedetect::detectObjects( MySize minSize, float scaleFactor, int
minNeighbors, int shift_step)
{
    /*grupisanje prozora koji se preklapaju*/
    const float GROUP_EPS = 0.4f;
    int y_bias, iter_counter = 0;

    /*faktor skaliranja*/
    float factor;

    /*velicina prozora kod trening seta*/
    MySize winSize0 = cascadeObj.orig_window_size;

    /*iteracija se vrši po piramidi slika*/
    face_number = 0;
    for( factor = 1.0f; ; factor *= scaleFactor)
    {
        /*brojac iteracija*/
        iter_counter++;

        /*upscaled velicina slike*/
        MySize winSize = { myRound(winSize0.width*factor),
myRound(winSize0.height*factor) };

        /*downscaled velicina slike*/
        MySize sz = { ( IMAGE_WIDTH/factor ), ( IMAGE_HEIGHT/factor ) };

        /*ako je velicina slike manja od velicine prozora, izađi iz
petlje*/
        if( sz.width < 24 || sz.height < 24 )
            break;

        /*ako je specificirana minimalna velicina prozora drugacija od
originalnog prozora, nastavi na sljedece skaliranje*/
        if( winSize.width < minSize.width || winSize.height <
minSize.height )
            continue;

        /*Racunski intenzivan korak:
izgradnja piramide slike downsamplingom slika
downsampling se vrši upotrebom nearest neighbor algoritma*/

        nearestNeighbor( downsample_buffer, sz.width, sz.height);

        /*Racunski intenzivan korak:
na svakom nivou piramide slika,
napravi novu integralnu i kvadriranu integralnu sliku
*/
        integralImages(downsample_buffer, int_img_buffer, sq_int_buffer,
sz.width, 25);

        /*
```

```

        * Sumiranje piksela u Haarovom prozoru vrši se upotrebom četiri
        coska integralne slike
        * Ova funkcija nije računski intenzivna jer ne vrši nikakav račun
        * Sljedeća računski intenzivna funkcija jeste ScaleImage_Invoker
        */

        setImageForCascadeClassifier( int_img_buffer, sq_int_buffer,
        sz.width);

        for(y_bias=0; y_bias < sz.height-25+1; y_bias++){
            if(y_bias!=0)
                //pomjeri bafer integralne slike i azuriraj samo posljednji
red piksela
                integralimages_lastrow(downsample_buffer, int_img_buffer,
sq_int_buffer, sz.width, y_bias);

                /*obrada trenutne skale slike upotrebom kaskadnog filtra
                glavni proračuni vrše se u sklopu ove funkcije
                */

                ScaleImage_Invoker( factor, sz.width, shift_step, y_bias);
        }

    } /*kraj petlje, završena obrada svih skala u piramidi*/

    if( minNeighbors != 0 )
    {
        groupRectangles( minNeighbors, GROUP_EPS);
    }
}

```

Listing 7. Kodni listing funkcije detectObjects()

Funkcije koje se koriste za detekciju su sljedeće :

nearestNeighbor – pomoću nearestNeighbor algoritma downsampluje sliku

integralImages , integralimages_lastrow – vrše proračun integralnih i kvadriranih integralnih slika i tako brzo sumiraju određene oblasti na slici

evalWeakClassifier – funkcija koja vrši evaluaciju slabih Haarovih klasifikatora, u suštini je implementacija Haarovog filtra, predstavlja najinteresantniju funkciju za hardversku akceleraciju jer je samo jezgro algoritma

ScaleImage_Invoker – takođe akceleracijski interesantna funkcija, prevlači prozor sa skaliranjem slike kako bi detektovala lica i čuva njegove koordinate u slučaju da ga pronađe

predicate – poredi dva pravougaonika i određuje njihovu sličnost

partition - funkcija koja od niza labela treba da pronađe broj lica tako što iz labela nalazi roditeljsko lice i tako pravi stablo sve dok ne dođe do najstarijeg lica (root), broj root-ova je broj prepoznatih lica

groupRectangles - funkcija koja se koristi za grupisanje detektovanih pravougaonika lica bazirano na metodi klasterovanja i granici preklapanja (eps parametar)

runCascadeClassifier - funkcija koja provlači sliku kroz kaskadu rastuće kompleksnih klasifikacija (faza) i vrši detekciju lica

updatePvalue – ažurira integralnu i kvadratnu sliku i priprema ih za dalje proračune.

setImageForCascadeClassifier - funkcija koja priprema i podešava podatke potrebne za računanje Haarovih karakteristika potrebnih za rad Viola-Jones algoritma.

Na osnovu kodnog listinga specifikacije i izgleda funkcije `detectObjects`, evidentno je da funkcije `groupRectangles` i `partition` nisu interesantne za hardversku akceleraciju iz razloga što nisu računski intenzivne funkcije. Ove funkcije stoga neće biti razmatrane u odjeljku u kome se budemo bavili hardverskom akceleracijom. Posebnu pažnju u tom odjeljku posvetićemo funkciji `evalWeakClassifier`, koja se gledajući sami kod najvjerovatnije najčešće poziva, a što bi vremenska analiza koda trebala potvrditi.

4. Analize prije particionisanja

4.1 Vremenska analiza

Vremensko profajliranje rada sistema izvršeno je upotrebom **MATLAB Profiler** alata za inicijelnju, odnosno **Valgrind/Callgrind** alata za prevedenu specifikaciju. Rezultati MATLAB profajliranja uključeni su na Git repozitorijum u HTML formatu. Tabela pregled vremenske zahtjevnosti funkcija dat je u sljedećoj tabeli:

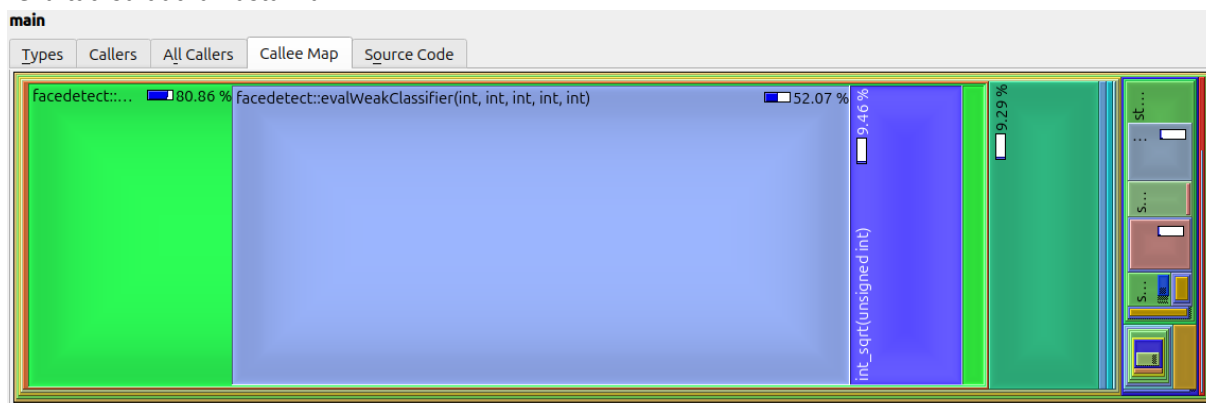
Profile Summary
Generated 15-Mar-2026 14:46:39 using performance time.

Function Name	Calls	Total Time	Self Time*	Total Time Plot (dark band = self time)
specifikacija	1	0.568 s	0.180 s	
...cadeObjectDetector.CascadeObjectDetector	1	0.190 s	0.045 s	
...tDetector>CascadeObjectDetector.stepImpl	1	0.156 s	0.113 s	
SystemProp.setProperties	1	0.112 s	0.015 s	
SystemProp.parseInputs	1	0.091 s	0.011 s	
...deObjectDetector.set.ClassificationModel	1	0.079 s	0.026 s	
...cadeObjectDetector.loadXMLFromClassModel	2	0.075 s	0.065 s	
...or>CascadeObjectDetector.preprocessImage	1	0.042 s	0.023 s	
SystemProp.createVersionNumber	2	0.040 s	0.027 s	
...CascadeObjectDetector.convertToGrayscale	1	0.019 s	0.011 s	
strsplit	2	0.012 s	0.010 s	
CascadeObjectDetector>isSimMode	5	0.011 s	0.010 s	
...ector>CascadeObjectDetector.getModelPath	2	0.009 s	0.002 s	
rgb2gray	1	0.007 s	0.007 s	
isSystemObject	1	0.007 s	0.007 s	
fullfile	4	0.006 s	0.004 s	
...CascadeObjectDetector.validateInputsImpl	1	0.002 s	0.001 s	
join	2	0.001 s	0.001 s	
fullfile>ensureTrailingFilesep	4	0.001 s	0.000 s	
isprop	1	0.001 s	0.001 s	
strfun\private\strescane	6	0.001 s	0.001 s	

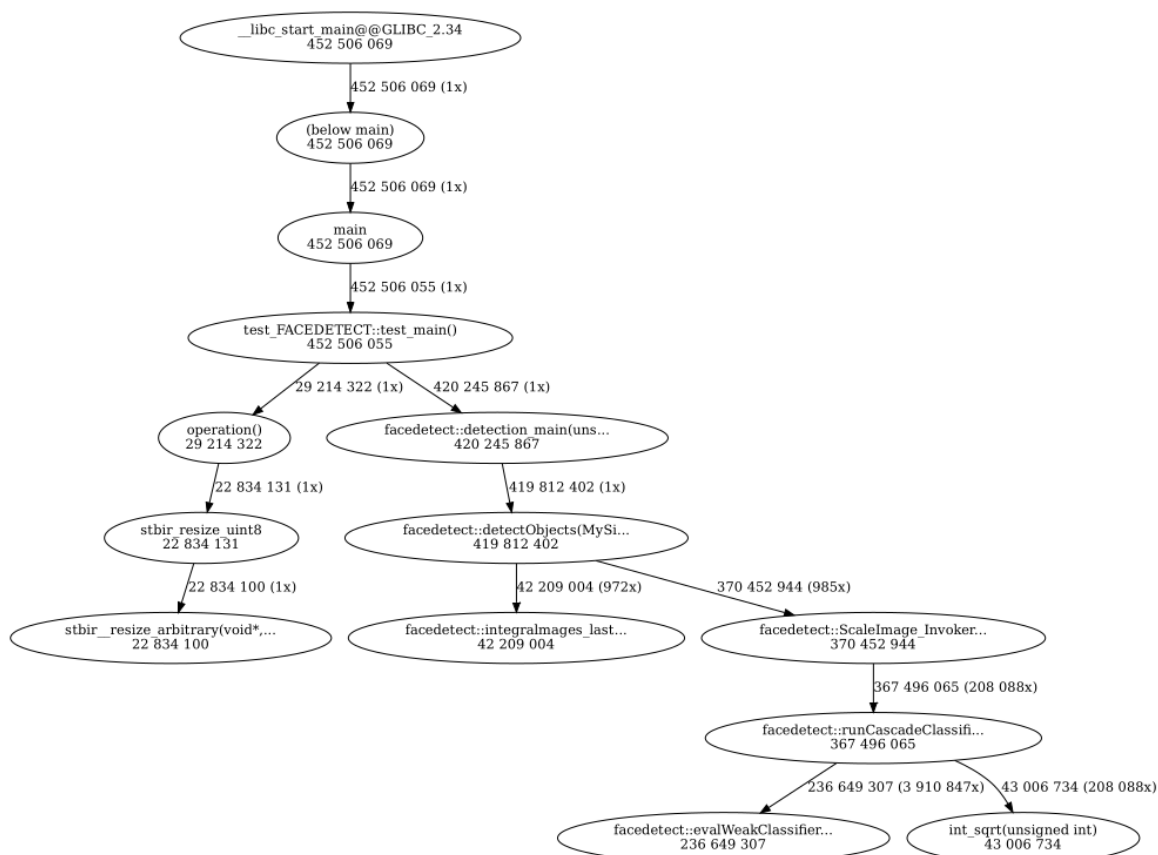
Tabela 2. Vremenska zahtjevnost funkcija dobijena u MATLAB profajleru

Tabela potvrđuje ono što smo već zaključili, a to je da će funkcije koje će se akcelerirati zapravo biti dijelovi Viola-Jones algoritma. Za detaljniju analizu korišćen je Valgrind/Callgrind profajler, čiji

rezultati su dati u nastavku :



Slika 9. Grafički prikaz Callgrind rezultata profajliranja u KcacheGrind alatu



Slika 10. Stablo poziva specifikacije prikazano u KcacheGrind alatu

Tabelarno, rezultati se mogu prikazati na sljedeći način (funkcije s manje od 1% doprinosa nećemo uzimati u obzir):

Funkcija	Procenat u broju ukupnih instrukcija
----------	--------------------------------------

evalWeakClassifier	52.07%
runCascadeClassifier	17.41%
int_sqrt	9.46%
integrallImages_lastrow	9.29%
updatePvalue	1.92%
stbir_resample_horizontal_downsample	1.24%
stbir_encode_scanline	1.14%

Tabela 3. Funkcije sa vremenski najzahtjevnijim kodom

Na osnovu grafa poziva i priložene tabele vidimo da je funkcija **evalWeakClassifier** funkcija koja se ubjedljivo ističe kao kandidat za hardversku akceleraciju. Ogroman broj poziva koji je ubjedljivo veći od bilo koje druge funkcije, kao i procenat u ukupnom broju instrukcija unutar same funkcije, nam garantuje da je to u pitanju kritična funkcija.

4.2 Algoritamska analiza

U ovom odjeljku biće urađeno poređenje treniranih modela po njihovim osobinama i objašnjeno zašto su određeni modeli u ovakvom sistemu bolji ili gori. Konkretno, specifikaciji su priključena dva pre-trained modela koji su standardan dio OpenCV biblioteke, 'haarcascade_frontalface_default.xml' i 'haarcascade_frontalface_alt_tree.xml'. Međusobno poređenje performansi pre-trained modela na testnom setu od 20 različitih fotografija sa različitim uslovima osvjetljenja i položaja lica dato je u sljedećim tabelama :

Haar Cascade FrontalFace default				
Ime slike	Tačan broj lica	Tačno detektovana lica	Netačno detektovana lica	
			Nedetektovana lica	Detekcija nepostojećeg lica
test1	26	24	2	1
test2	2	2	0	7
test4	11	11	0	0
test5	11	11	0	4
test6	11	11	0	16
test7	36	31	5	6
test8	30	30	0	1
test10	7	7	0	2
test11	18	18	0	11
test12	6	6	0	0
test13	5	5	0	0
test14	5	5	0	2
test15	5	3	2	0
test16	9	8	1	1
test17	23	23	0	4
test18	8	8	0	19
test19	16	13	3	1
test20	5	4	1	0
	234	220	14	75

		94,02%	5,98%	32,05%
--	--	--------	-------	--------

Tabela 4. Performanse modela Haar Cascade FrontalFace default

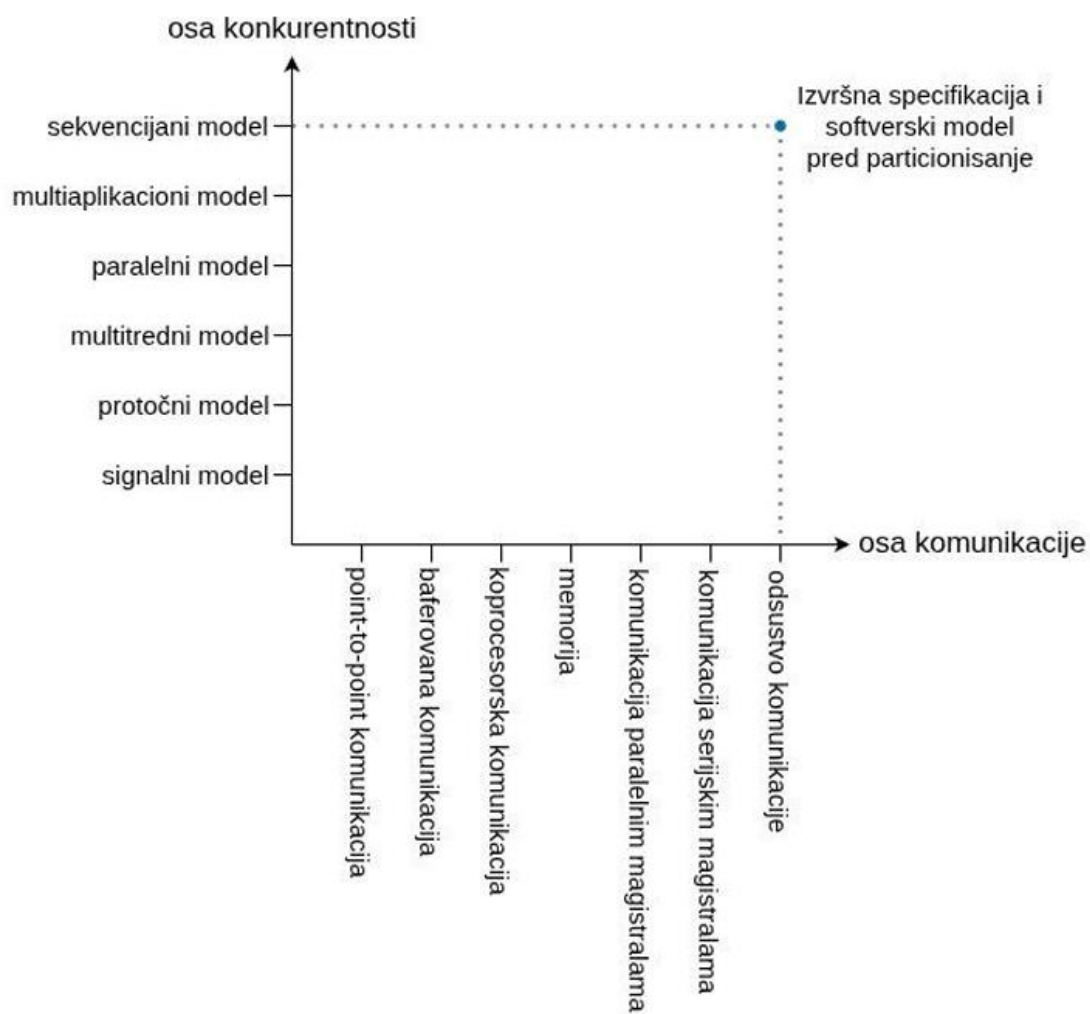
Haar Cascade FrontalFace alt tree				
Ime slike	Tačan broj lica	Tačno detektovana lica	Netačno detektovana lica	
			Nedetektovana lica	Detekcija nepostojećeg lica
test1	26	6	20	0
test2	2	0	2	0
test4	11	4	7	0
test5	11	11	0	0
test6	11	9	2	0
test7	36	20	16	0
test8	30	23	7	0
test10	7	6	1	0
test11	18	17	1	0
test12	6	5	1	0
test13	5	3	2	0
test14	5	3	2	0
test15	5	2	3	0
test16	9	1	8	0
test17	23	19	4	0
test18	8	8	0	0
test19	16	8	8	0
test20	5	2	3	0
	234	147	87	0
		62,82%	37,18%	0,00%

Tabela 5. Performanse modela Haar Cascade FrontalFace alt tree

Na osnovu rezultata i tabela može se zaključiti da default model nudi veći recall (veći broj detektovanih lica), ali kao posledicu ima i veći broj lažno pozitivnih detekcija, što je djelimično i posledica algoritma, dok alt_tree model nudi veću preciznost pri detekciji, ali ima znatno manji recall od default modela. U većini situacija i primjena gdje je detekcija lica potrebna (npr. nadzorni sistemi) uvijek je bolje imati veći recall, jer se lažno pozitivne detekcije uz intervenciju korisnika mogu odbaciti kao sistematske greške i neće imati značajan negativan uticaj na korisnika. Stoga je pri projektovanju ovakvog sistema između dva modela ovih karakteristika bolje odabrati default model, jer prednosti upotrebe takvog modela značajno nadmašuju mane istog.

Na osnovu do sada izvršenih analiza, zaključujemo da je, taksonomski, naš model ostao isti, to jeste naš model pred particionisanje se nalazi u istom položaju na osi konkurentnosti i osi komunikacije kao

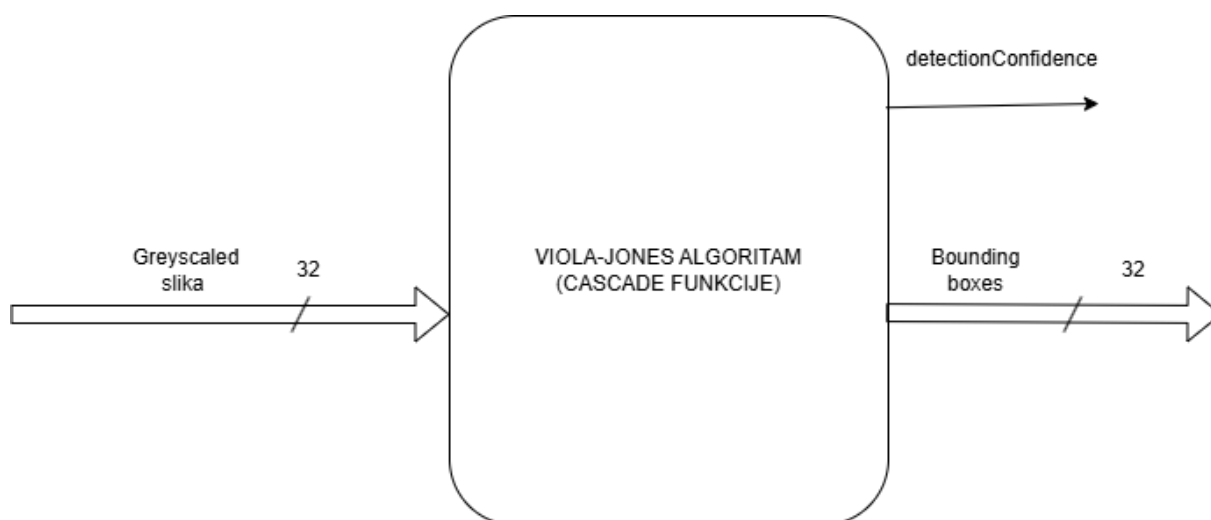
što je to prethodno i teorijski utvrđeno :



Slika 11. Položaj softverskog modela na osama ESL taksonomije pred particionisanje

4.3 Bitska analiza

Bitsku analizu nad Viola-Jones algoritmom vršićemo virtuelno, a kasnije će biti dopunjena i simulacionim rezultatima. Blok šema koja predstavlja Viola-Jones algoritam sa svojim ulazima i izlazima data je na slici ispod:



Slika 12. Prikaz Viola-Jones algoritma kao prostog bloka sa I/O interfejsom od interesa

Viola-Jones algoritam kao ulaz prima 8-bit greyscale sliku. Razlog zašto je signal širine od 32 bita leži u tome što je za reprezentaciju slike rezolucije 360*240 (ulazna rezolucija slike koje ćemo koristiti) u 8-bit greyscaleu potrebno $360 \cdot 240 \cdot 1 = 86,400$ bajta ili približno 84.38 KB. U greyscaleu svaki piksel nosi tačno jedan bajt podataka i može imati vrijednosti od 0 do 255. Šira magistrala nije obavezna za prenos greyscale slike, jer svaki piksel ima samo 8 bita, ali korišćenje šire magistrale (poput 32-bitne) omogućava prenos više piksela odjednom, što povećava brzinu prenosa.

Na 8-bitnoj magistrali: Prenosi se **1 piksel po ciklusu**.

Na 32-bitnoj magistrali: Prenose se **4 piksela po ciklusu**.

Zaključak je da se slika kao ulaz mora prenositi minimalno kao 8-bitan podatak, daljim povećanjem bitwidtha dobićemo na brzini prenosa u zamjenu za veću iskorištenost resursa, a daljim smanjivanjem nećemo imati tačan podatak na izlazu.

Analogno tome isti zaključak možemo izvesti i za **boundingBoxes** izlaz, koji predstavlja identičnu greyscaled sliku sa ucrtanim graničnim pravougaonicima u oblastima gdje su detektovana lica.

Zanimljiva polemika može se provesti nad signalom detectionConfidence. U Viola-Jones algoritmu, izlaz detectionConfidence predstavlja **stepen sigurnosti** algoritma u vezi sa detektovanjem objekta na određenoj lokaciji u slici. To je numerička vrijednost koja se koristi za procjenu koliko je vjerovatno da je detektovani region zaista traženi objekat (npr. lice). Predstavlja vrijednost u rasponu od 0 do 1 sa određenim pragom (threshold) za koji je algoritam siguran da je detektovani objekat na slici lice. Obično se predstavlja float ili double tipom, pa ga za potrebe izvršavanja na FPGA i lakše implementacije moramo prebaciti u fixed point aritmetiku. To se može uraditi na sljedeći način:

Pretpostavimo da imamo vrijednost detectionConfidence = 0.75 i želimo da je konvertujemo u unsigned 16-bitni fixed-point integer.

1. Faktor skaliranja: $2^{16}=65536$
2. Pomnožimo sa faktorom: unsigned-point vrijednost= $0.75 \cdot 65536=49152$
3. Dobijeni rezultat 49152 je sada unsigned fixed-point reprezentacija vrijednosti 0.75.

Očigledno, zavisno od toga koji tip odaberemo, mijenjaće nam i preciznost detekcije. Ukoliko nam je prag detekcije 0.95, vrijednost može biti **16-bitni fixed-point** int ili **32-bitni fixed-point** int, od kojih svaki ima svoje prednosti i mane :

16-bitni unsigned fixed-point je pogodniji za manju preciznost i veću memorijsku efikasnost, sa opsegom od 0 do 1.0 sa tačnošću do 16 decimala.

32-bitni unsigned fixed-point je precizniji (24 bita za decimalnu tačku), ali zahtijeva više memorije.

U našem slučaju, možemo se odlučiti za 16-bitni unsigned fixed-point.

Kod korišten za realnu bitsku analizu (konkretno nad našom specifikacijom) uključen je u izvorne fajlove na repozitorijumu. Razlikuje se od virtuelne analize po tome što smo u virtuelnoj analizirali pretpostavljene ulaze, ne razmišljajući o internim promjenljivima, dok je ovdje urađen sweep svih internih promjenljivih koje moraju raditi sa frakcionim dijelom, a koje su potencijalno problematične. Za testiranje je korišćen set od 10 slika, kao i tekst fajl koji sadrži tačnu vrijednost broja lica koje sistem treba detektovati. Sumirano, analiza tačnosti sistema, zavisno od vrijednosti sweepovanih promjenljivih sa pokretnim zarezom, prikazana je na sljedeći način :

Varijabla	Teorijski optimalni tip	Stepabilno ?	Minimalna radna širina
scaleFactor	sc_ufixed<8,1>	NE	optimalna
factor	sc_ufixed<10,5>	NE	optimalna
eps / GROUP_EPS	sc_ufixed<8,1>	NE	optimalna
delta (predicate)	sc_ufixed<16,8>	NE	optimalna
s (1/rweights)	sc_ufixed<10,1>	NE	optimalna
myRound argument	sc_ufixed<16,12>	DA	16 (8 I, 8F)

Tabela 6. Uticaj stepovanja promjenljivih unutar facedetect.cpp na tačnost

Iz tabele zaključujemo sljedeće:

Promjenljive čija je stepabilnost označena sa NE (scaleFactor, factor, eps, delta i s) su promjenljive čije su vrijednosti cijelog i frakcionog dijela **teorijski strogo definisane**. Svako odstupanje od ovih vrijednosti, pa makar bilo i minimalno, ima ogroman uticaj na tačnost rezultata (u svim situacijama, broj detektovanih lica je pao na 0). Jedina promjenljiva čija se raspodjela bitova može mijenjati, bez značajnijeg uticaja na tačnost, jeste argument iz myRound funkcije. Kod njega, smanjenje integer dijela značajno utiče na performanse, dok je uticaj frakcionog dijela zanemarljivo mali. Kod promjene broja cijelih bitova, postoji očigledna stepenasta funkcija u ponašanju. Poređenje sa float referencom u predstavljanju brojeva nakon stepovanja nalazi se ispod:

$I_RND \leq 4$ - F1 = 0.000 (potpuni krah)

$I_RND = 5$ - F1 = 0.353

$I_RND = 6$ - F1 = 0.473

$I_RND = 7$ - F1 = 0.817

$I_RND \geq 8$ - F1 = 0.923 (maksimum, float referenca)

Zaključujemo da se myRound optimalno može definisati kao sc_ufixed<16,8> tip, što je za FPGA odlično, jer je tačno jedan bajt cijeli, a jedan razlomljeni dio. Kritična funkcija **evalWeakClassifier ne**

radi s float vrijednostima, pa stoga nema poente za posebnom analizom iste što se toga tiče.

Međutim, kao što ćemo vidjeti u odjeljku u kome smo objasnili partitionisanje, odabir dijela koji akceleriramo značajno može uticati na podatke koji su nam relevantni iz bitske analize.

5. Partitionisanje

Partitionisanje ovog sistema predstavlja veoma bitan i izazovan proces, kako bismo znali koji dio sistema akceliramo. U ovom poglavlju se objašnjava virtuelna platforma za razvoj softvera na procesoru ZYNQ-7000 SoC koji se nalazi na pločici Zybo Z7-10. Na osnovu svih prethodnih podataka i zaključaka, možemo zaključiti da će se akceleracija vršiti nad dijelovima algoritma odgovornim za detekciju lica. Ideja partitionisanja jeste sljedeća:

HARDVER: dio koji se odnosi na detekciju lica (Viola-Jones IP blok), minimalno `evalWeakClassifier` funkcija. **Zbog jednostavnosti predviđene funkcije i njene interakcije sa kaskadnom klasifikacijom, koja bi u slučaju samo njenog akceleriranja, značila ogroman broj transakcija koji bi znatno usporio rad čitavog sistema, u nastavku je predviđeno akceleriranje čitave `runCascadeClassifier` funkcije, pa je u nastavku to naglašeno !**

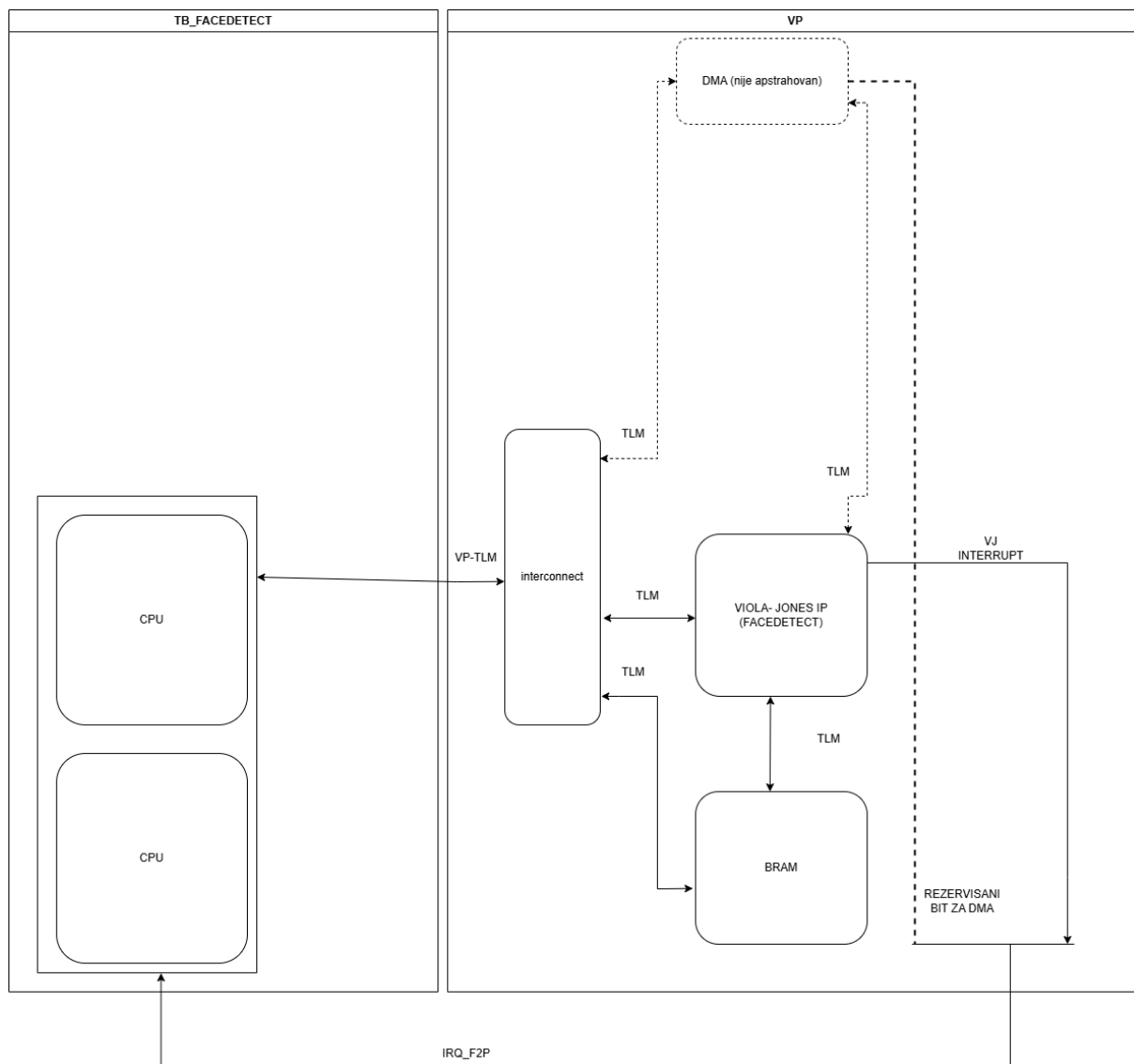
SOFTVER: preprocesing i postprocesing operacije potrebne za ispravan rad uređaja (resizing ulazne slike `input.jpg` na veličinu potrebnu za rad detektora, greyscaling tako obrađene slike, povratni ispis na terminal). Nakon kompajliranja rad sistema se inicira pozivom `./facedetect.exe` u terminalu, kao i do sada.

5.1 Virtuelna platforma

Teorijski, ovo bi bilo idealno partitionisano, međutim u praksi je, zbog ograničenih resursa na FPGA, potrebno napraviti određena kompromisna rješenja pri odabiru funkcije koju hardverski akceliramo, jer je zbog ograničene BRAM memorije teško akcelirati čitav algoritam na Zybo Z7-10 pločici, pa se samim time i partitionisanje mijenja. Iz vremenske analize primjetno je da funkcija `evalWeakClassifier` zauzima najviše vremena izvršavanja, dok je funkcija `runCascadeClassifier` odmah iza nje po tom parametru. Na osnovu toga, zaključujemo da su te dvije funkcije najočigledniji kandidati za akceleraciju, koje je na odgovarajući način potrebno prilagoditi tome, pa ćemo se time baviti u nastavku.

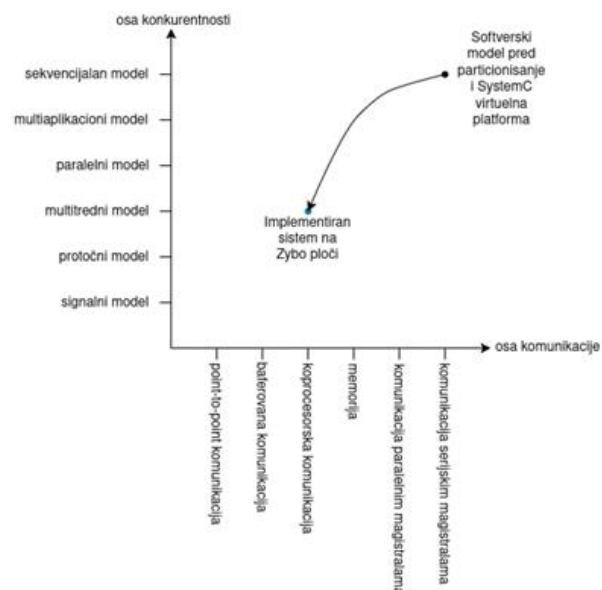
Ovako partitionisana virtuelna platforma je sposobna da modeluje našu hardversko-softversku spregu koju očekujemo u sistemu.

To se može prikazati na sljedeći način:



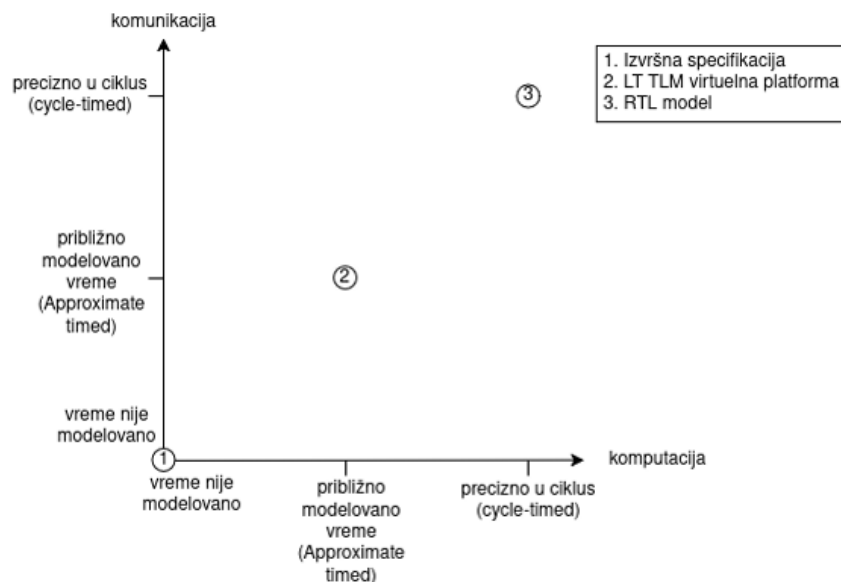
Slika 13. Pregled virtuelne platforme

Prilikom modelovanja ovog sistema, korišćen je TLM 2.0 standard za povezivanje. Unutar virtuelne platforme, kasnije i implementacije, kritična funkcija, odnosno akcelerirani dio sistema se modeluje IP modulom i time se teorijski u taksonomskom smislu sistem pretvara u **multinitan sistem sa koprocesorskom komunikacijom**.



Slika 14. Izgled partitionisanog sistema nakon implementacije na osama taksonomije

Virtuelna platforma je modelovana **LT** stilom pomoću **SystemC** biblioteke. Ova C++ biblioteka nam omogućava da simuliramo hardver na nivou diskretnih događaja. Koristi se zajednički model hardvera i softvera u OSCI simulatoru. Prilikom klasifikacije ovog modela po vremenskoj osi, može se govoriti o događajima koji su **uzročno - posljedično povezani**.



Slika 15. Roadmap pozicioniranja sistema na osama komunikacije i proračuna

Model se nije mijenjao po osi konkurentnosti i konfigurabilnosti u odnosu na model pred partitionisanje. Zbog korišćenja **LT** stila, sistem je **sekvencijalan**.

Na samom modelu virtuelne platforme, primjetan je dio gdje je naglašen rezervisani bit za DMA. **DMA (Direct Memory Access) modul nije potreban na ovom dijelu projekta**, jer je slika konstantna i

ne ažurira se kontinualno, ali će biti implementiran u okviru predmeta **Projektovanje složenih digitalnih sistema**, zajedno sa paralelizacijom klasifikacije, kako bi se čitav proces rada ubrzao (konkretno za DMA, upis generisanih integralnih slika u baferu u DDR3 memoriji) i time dobile bolje performanse. Rad bez DMA modula je moguć, ali je rezervisani bit ostao kao smjer za dalje unapređivanje sistema.

Paralelizam u ovom sistemu odvija se preko SystemC procesa, odnosno njihove međusobne konkurentnosti. Najvažniji procesi koji se uporedo odvijaju jesu sljedeći:

Niti SC_THREAD core0 i SC_THREAD core1 se startuju u istom vremenskom trenutku (oba u delta 0) – core0 radi na preprocessing obradi slike, dok core1 čeka na spremnu/obrađenu sliku.

Dok nit core1 čeka na done događaj, interrupt sekvenca (koja se sastoji od `gic()` SC_METHOD i `IP_ISR()` SC_THREAD) **se neprekidno izvršava u istom tom vremenskom okviru**, što je pravi primjer paralelizma.

Metoda za konkatenaciju bitova SC_METHOD vp::concat() se izvršava uporedo sa svim ostalim procesima – kada god IP podigne interrupt signal, ona se izvršava, bez obzira na to šta se dešava u ostatku sistema. Iako na prvi pogled ne djeluje tako, ona zapravo modeluje paralelizam, jer paralelno sa svim ostalim operacijama u sistemu konkatenira bitove prekida IP bloka i DMA modula. Kao što smo već zaključili, DMA modul u implementaciji na ovom predmetu nije apstrahovan, ali će biti prisutan na RTL dijelu kao jedan od načina da se rad čitavog sistema ubrza, **međutim treba biti pažljiv i posmatrati ga kao jedan od elemenata konkatenacione metode, kako bi se kasnije ispravno implementirao na RTL dijelu projekta.**

Sveukupno, paralelizam na ovom dijelu projekta je ograničen, dok će detaljnija paralelizacija sistema biti sprovedena na RTL dijelu kako bi se uspješno izvršila akceleracija sistema.

Model koji imamo kada u virtuelnoj platformi iz postojeće specifikacije kreiramo module i izmodelujemo testbench tako da radi sekvencijalno i radi „polling“ na određene signale i na osnovu toga izvršava algoritam jeste **sekvencijalni model sa komunikacijom serijskim magistralama** na ESL osi taksonomije, što očigledno nije ciljni model, ali smo u njemu jasno definisali izgled budućih modula te njihove međusobne komunikacije. Nakon reparticionisanja i analize u kojoj smo zaključili da će struktura IP-a biti nešto drugačija od prvobitno planirane, dobili smo sljedeće module :

Procesor

tb_vp fajlovi modeluju dvojezgarni ARM procesor koji možemo pronaći na Zybo Z7-10 ploči. Pošto je ciljani model **multinitnog sistema sa koprocesorskom komunikacijom**, svako jezgro predstavlja zasebnu nit koja se „budi“ kada se desi određeni sinhronizacioni događaj. Svako jezgro ima svoju funkciju, za ovaj sistem konkretno to su sljedeće:

core0 : Interakcija sa spoljnim svijetom oko virtuelne platforme: učitavanje slike iz fajla, čekanje rezultata detekcije i povratni ispis u terminalu.

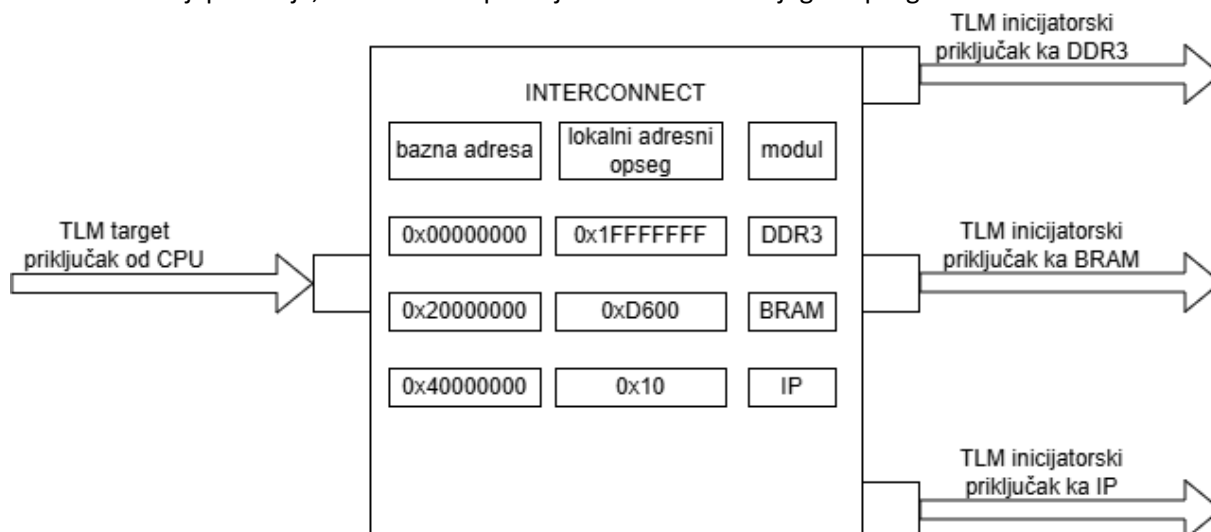
core1 : Interakcija sa hardverom na virtuelnoj platformi : koordinacija algoritma detekcije lica - računanje integralnih slika, upisivanje u DDR3, pokretanje IP-a po detekcijskom prozoru, grupisanje preklapljenih pravougaonika i kreiranje rezultata.

Dodatno, u procesoru je modelovan GIC (Generic Interrupt Controller) kao metoda koja se okida na svaku promjenu **IRQ_F2P** signala, koji demultipleksuje 2-bitni signal i notifikuje odgovarajući događaj,

gdje je nulti bit indikator prekida na IP-u, kao i prekidna rutina za IP koja potvrđuje DONE bit i notifikuje core1 da može čitati rezultat. Izgled komunikacije prekidima u okviru ove virtualne platforme diskutovaćemo u nastavku.

Interconnect

Interconnect modul je komponenta koja povezuje DDR3 RAM, IP i BRAM modul i omogućava procesoru da im pristupa preko jednog standardnog TLM priključka. Moduli su memorijski mapirani, odnosno postoji jedinstveni adresni opseg unutar kojeg svaki modul dobija jedinstvenu baznu adresu i opseg lokacija koji pokriva. Ovim moduli, sa stanovišta procesora predstavljaju samo lokacije u toj jedinstvenoj memoriji. Na lokacijama dobijenim sabiranjem bazne adrese modula i lokalne adrese se nalaze SAHE elementi (eng. **Software Accessible Hardware Elements**) modula koji su interconnect modulom objedinjeni u virtualnu platformu. U trenutku kada procesor šalje TLM paket ka SAHE elementu određenog modula, on šalje paket ka interconnect modulu na adresu jedinstvenog adresnog opsega. Uloga interconnect modula je da na osnovu adrese paket usmjeri ka ciljnom modulu, tako što pretvori adresu jedinstvenog adresnog opsega u lokalnu adresu ciljnog modula. Za svaki modul koji povezuje, interconnect poznaje baznu adresu i njegov opseg lokalnih adresa.



Slika 16. Interconnect modul

BRAM

BRAM memorijski modul skladišti klasifikacione podatke kojima se najčešće pristupa, i to su redom:

- 1. rectangles_array:** 34956 **char** vrijednosti, sadrži podatke o geometriji Haarovih karakteristika (x, y, w, h za svaki pravougaonik), lokalna adresa: VP_ADDR_BRAM_RECT (= 0).
- 2. stages_array:** 25 **uint8** vrijednosti, svaka lokacija sadrži podatak o broju slabih klasifikatora po fazi kaskade, lokalna adresa: VP_ADDR_BRAM_STAGES.
- 3. stages_thresh_array:** 25 **short int** vrijednosti, svaka lokacija je prag za odlučivanje na nivou faze, lokalna adresa: VP_ADDR_BRAM_STAGES_TH.
- 4. weights_array:** 8739 **short int** vrijednosti, lokacije čuvaju težine Haarovih karakteristika, lokalna adresa: VP_ADDR_BRAM_WEIGHTS.
- 5. alpha1_array:** 2913 **short int** vrijednosti, lokacije su koeficijenti slabih klasifikatora (lijeva grana), lokalna adresa: VP_ADDR_BRAM_ALPHA1.
- 6. alpha2_array:** 2913 **short int** vrijednosti, koeficijenti slabih klasifikatora (desna grana), lokalna

adresa: VP_ADDR_BRAM_ALPHA2.

7. tree_thresh_array: 2913 short int vrijednosti, pragovi odlučivanja po slabom klasifikatoru, lokalna adresa: VP_ADDR_BRAM_TREE_TH.

Svi ovi podaci su **read-only** i ne mogu se mijenjati, odnosno predstavljaju podatke dobijene treniranjem modela. Zbog ograničenosti resursa, ovo su jedini podaci koji se nalaze u BRAM-u, u slučaju realizacije sistema na hardveru sa većim BRAM-om dodatni kandidati za BRAM bili bi baferi za generisanje integralnih slika, koji su u ovoj realizaciji smješteni u DDR3 memoriju i read/write su tipa, ali bi to zahtijevalo izmjene u konceptu načina na koji virtuelna platforma funkcioniše pa se zato time nećemo baviti.

Na narednoj slici je prikazana konfiguracija BRAM modula i način na koji su podaci raspoređeni u njemu.

BRAM	
lokalna adresa	vrijednost
0	rectangles_array[0]
34956	stages_array[0]
34981	stages_thresh_array[0]
35006	weights_array[0]
43745	alpha1_array[0]
46658	alpha_2_array[0]
49571	tree_thresh_array[0]
52483	tree_thresh_array[2912]

Slika 17. Konfiguracija i raspored podataka u BRAM modulu

DDR3

Funkcionalnost sekvencijalne DDR3L memorije jeste čuvanje rezultata rada generatora integralnih slika. Prvobitna ideja da generisanje integralnih slika bude dio hardvera nije implementirana zbog

nedostatka prostora za smještanje svih radnih bafera generatora u BRAM, pa je stoga ovaj proces smješten u DDR3 memoriju. U njoj se nalaze dva velika radna bafera :

int_img_buffer : **25 * IMAGE_WIDTH** int vrijednosti (čuva podatke o integralnoj slici, upisuje ih jezgro core1 za svaki y_bias korak).

sq_int_buffer : **25 * IMAGE_WIDTH** int vrijednosti (kvadrirana integralna slika, upisuje core1 svaki y_bias korak).

Sadržaj ovih bafera se u memoriji čuva kao jedinstveni **mem[]** niz. IP ove bafere čita direktno tokom klasifikacije prozora, a core1 ih upisuje nakon svake iteracije generisanja integralne slike (što se dešava promjenom y_bias koraka prozora). Svaki od ovih bafera sadrži po 9000 lokacija u memoriji ukupne veličine od oko 72 KB. Na narednoj slici dat je pregled DDR3 memorije i rasporeda podataka u istoj.

DDR3	
lokalna adresa	vrijednost
0	mem[0] - prva lokacija bafera integralne slike
8999	mem[8999] - posljednja lokacija bafera integralne slike
9000	mem[9000] - prva lokacija bafera kvadrirane integralne slike
17999	mem[17999] - posljednja lokacija bafera kvadrirane integralne slike

Slika 18. Konfiguracija i raspored podataka u DDR3 modulu

IP modul

Preko **cfg_reg** SAHE elementa procesor upravlja IP modulom. U trenutku kada procesor izgeneriše integralnu sliku i inicijalizuje memoriju, proces klasifikacije u IP modulu može da počne. Prvi korak je da se modul resetuje upisom jedinice na nulti bit **cfg_reg** registra. Resetom se svi njegovi biti postavljaju na nulu. Sa procesom klasifikacije se započinje upisom jedinice na prvi bit registra. Modul vrši klasifikaciju i rezultat klasifikacije smješta u **result_reg** registar, koji može da primi **int32** vrijednost i klasifikacija kao rezultat vraća 1 ako je detektovano lice, odnosno -i ako je detekcija završila na i-toj fazi. Klasifikacija u IP-ju modeluje funkciju **runCascadeClassifier** iz originalnog koda za jedan prozor i ona se sastoji iz sljedećih koraka: za svaki poziv (jedan prozor):

1. Čita 8 vrijednosti iz DDR3 (updatePvalue pomoćna funkcija).

2. Računa `variance_norm_factor`.

3. Za svaku od 25 faza:

a. Čita `stages_array[i]` iz BRAM.

b. Za svaki slab klasifikator:

- čita `tree_thresh` iz BRAM.

- čita 4 char vrijednosti (x,y,w,h) iz BRAM za svaki pravougaonik.

- računa linearne ofset indekse (funkcionalnost `scaled_rectangles` iz specifikacionog koda).

- čita `int_img` vrijednosti iz DDR3.

- čita `weight` iz BRAM.

- akumulira `stage_sum`.

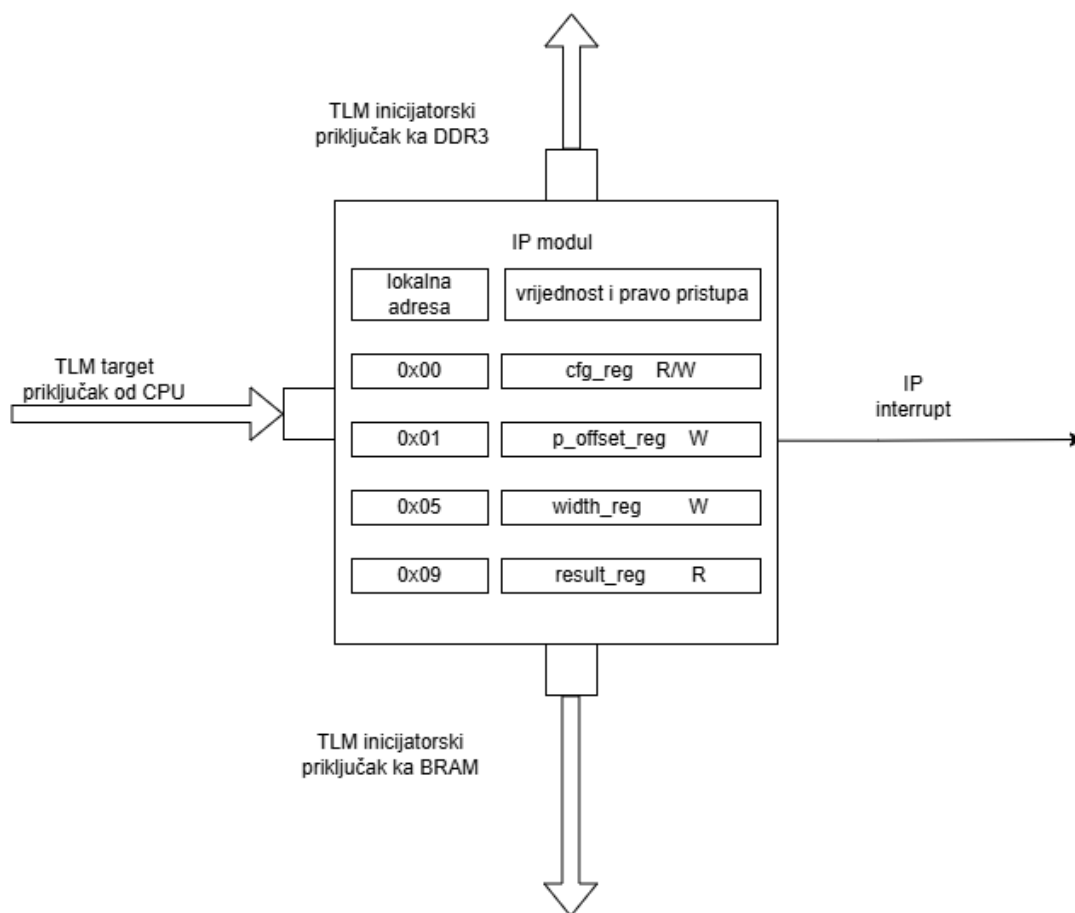
c. Čita `stages_thresh` iz BRAM.

d. Poredi `stage_sum` sa pragom (kraj klasifikacije ako klasifikator ne prođe prag).

4. Upisuje `result_reg`, setuje `DONE` bit, podiže `ip_interrupt` na visoku vrijednost.

Kada se vrijednost registra **result_reg** ažurira, odnosno dostigne se kraj procesa klasifikacije, četvrti bit registra **cfg_reg** se postavlja na jedinicu i interrupt izlaz se postavlja na visoku vrijednost, čime se procesoru javlja da je klasifikacija završena i da se vrijednost **result_reg** registra može pročitati.

Primjetno je da su treći i drugi bit konfiguracionog registra neiskorišteni, ali o njihovoj rezervisanosti će više biti riječi u poglavlju Registarske mape.



Slika 19. IP modul

Komunikacija u ovom sistemu modelovana je preko sistema interruptova (prekida) . Ovakav način komunikacije se odvija na sljedeći način:

1. IP klasifikator se startuje tako što **core1** upiše startni bit u konfiguracioni registar IP modula.
 2. Blok transport metoda unutar IP-a prima upis od interconnect modula, kopira ga u konfiguracioni registar i šalje notifikaciju da se stanje registra ažuriralo.
 3. **ip_thread** se budi, čita stanje konfiguracionog registra, nakon što pročita visoki startni bit preko bitmaske, ulazi u START stanje i otpočinje klasifikaciju.
 4. Klasifikacija se izvršava, nakon njenog završetka i dobijanja rezultata, DONE bit se setuje, START bit se resetuje, a na **ip_interrupt** izlaz se upisuje visoka vrijednost.
 5. **concat()** metoda je osjetljiva na ip_interrupt signal – ona se poziva i pakuje ip_interrupt signal na nulti bit dvobitnog **interrupt** izlaza (gornji bit je rezervisan za DMA, kao što smo prethodno ustanovili).
 6. Preko signala **IRQ_F2P** se te promjene prenose u **tb_vp** (CPU).
 7. **gic()** metoda je osjetljiva na IRQ_F2P signal – ona se poziva, pročita najniži bit, i u slučaju njegove postavljenosti notifikuje sistem da se desio IP prekid.
 8. **IP_ISR** nit čeka događaj na ip_interrupt signalu. Kada ga gic notifikuje, nit se budi, čita konfiguracioni registar preko TLM-a i provjerava DONE bit.
 9. U slučaju da je DONE bit postavljen, on ga resetuje i notifikuje sistem da je u pitanju DONE događaj na IP-u.
 10. Izvršavanje core1 jezgra je blokirano sve ovo vrijeme dok god se ne desi DONE događaj na IP-ju. Nakon što ga prekidna rutina IP-a notifikuje o tome da se događaj desio, core1 se budi, pročita rezultat klasifikacije, i nastavlja sa daljim izvršavanjem.
- Iako je sistem SystemC simulacija (još uvijek softverski model, bez HLS ili RTL implementacije), na ovaj način je modelovana **koprocesorska i memorijska komunikacija**, koja će profiniti dosadašnje odsustvo komunikacije čisto softverskog modela.

5.2 Registarske mape

Registri od interesa koji su predmet ovog poglavlja nalaze se u IP modulu. Osnovni **SAHE** element koji ćemo detaljno analizirati jeste **cfg_reg** i on će biti mapiran na 8-bitni registar istog imena. Softver je modelovan tako da je najmanji podatak koji može biti adresiran veličine jednog bajta (eng. **Least Adressable Unit - LAU**). Raspored pojedinačnih bitova i njihovo značenje je dato slikom u nastavku. Registar se koristi tako što procesor pročita vrijednost registra (prva transakcija). Na osnovu trenutne vrijednosti pomoću bitmaske procesor podešava odgovarajuće bite registra (druga transakcija) i tako komanduje procesima klasifikacije i čitanja rezultata iz IP-ja. Softverski API čine prosti upis/čitanje odgovarajućih vrijednosti iz registra. Sporedni efekti (**side effects**) nisu prisutni.



Slika 20. Mapa cfg_reg registra

Značenje DONE, START i RESET bitova objašnjeno je u prethodnom dijelu dokumentacije. Kao što smo već rekli, bitovi 2 i 3 su rezervisani za rad sa DMA blokom, kojeg smo izostavili na ovom dijelu projekta, ali koji će neminovno postati dio RTL implementacije ovog sistema. Bitovi 2 i 3 upravljaju zahtjevima ka DMA, odnosno DMA streamovima podataka, ali se time u ovom dijelu realizacije sistema nećemo baviti, međutim bitno je da znamo da postoje kako bismo znali kako da ih iskoristimo za RTL implementaciju sistema.

Ono što je takođe bitno napomenuti je da je cfg_reg registar **read/write** tipa pristupa, što znači da CPU može postavljati ili resetovati bite u njemu po želji, ali i čitati iste kako bi rukovodio radom sistema.

Dodatni registri koje moramo spomenuti a koji su dio IP modula su sljedeći:

p_offset_reg – čuva vrijednost horizontalne pozicije trenutnog klasifikacionog prozora unutar bafera integralne slike. Upisuje ga **core1** prije svakog startovanja detekcije. IP ga koristi kao bazni ofset unutar pomoćne funkcije **update_pvalue** za čitanja vrijednosti čoškova prozora iz DDR3 memorije, kao i za horizontalno pomjeranje prozora unutar funkcije **eval_weak_classifier**, i tako prevlači prozor klasifikatora na tačnu x poziciju unutar trenutnog reda integralne slike. Tipa je **uint32_t** i u njega procesor može isključivo upisivati (ima samo write pristup).

width_reg – čuva širinu downsamplovane slike u trenutnoj skali piramide integralnih slika. Upisuje ga **core1** jednom po skali slike. IP ga koristi unutar pomoćne funkcije **update_pvalue** kako bi izračunao vertikalno rastojanje između redova, a takođe se iz istog razloga koristi i u **eval_weak_classifier** funkciji. Bez poznavanja ovog parametra, IP ne bi mogao pretvoriti 2D koordinate u indeks memorijske lokacije unutar DDR3 memorije. Tipa je **uint32_t** i njemu takođe procesor ima samo write pristup.

result_reg – predstavlja izlaz IP modula, klasifikacioni rezultat trenutno obrađenog prozora. IP ga upisuje na kraju procesa klasifikacije. Ima dvije moguće vrijednosti : **1** ako je prozor prošao svih 25 faza kaskadne klasifikacije (što znači da je lice pronađeno) ili **-i** u slučaju da prozor nije prošao na i-toj fazi klasifikacije (npr. -15 znači da na 15-toj fazi klasifikacije prozor nije prošao prag). Ovaj registar **core1** jezgro čita nakon DONE događaja na interrupt signalu. Tipa je **int32_t** i procesor ga isključivo može čitati, ali ne i pisati u njega (ima samo read pristup, za write je zadužen IP).

Na ovaj način smo uspješno izvršili particionisanje, realizaciju virtuelne platforme i mapiranje registara u okviru ovog sistema. Ostalo je da se izvrši procjena parametara relevantnih za rad ovog uređaja u praksi, za šta će nam biti potrebne analize u okviru nekog HLS alata, koje će detaljnije biti objašnjene u poglavlju Analize poslije particionisanja.

6. Analize poslije partitionisanja

6.1 Analiza performansi

Za procjenu performansi izabrano je sintetizovati proces detekcije i kaskadne klasifikacije lica korišćenjem Vivado HLS alata. S obzirom na to da je partitionisanje izvršeno i postoji jasno definisan hardver, potrebno je napisati C++ izvorne fajlove koji modeluju proces kaskadne klasifikacije lica upotrebom Viola-Jones algoritma uz uvažavanje izvršenog partitionisanja. Deklaracija funkcije koja će se sintetizovati data je kodnim listingom u nastavku.

```
void top_function(ap_uint<8> *cfg_val, ap_uint<32> p_offset, ap_uint<32>
width, ap_int<16> bram[BRAM_SIZE], ap_int<32> ddr3[DDR3_TOTAL_SIZE], bool
*ip_interrupt, ap_int<32> *result);
```

Listing 8. Deklaracija top funkcije u HLS implementaciji

Parametri koje imamo u okviru ove funkcije modeluju registre IP bloka, prateće memorije i prekidne signale:

cfg_val je vrijednost koja će se upisivati u promjenljivu koja će modelovati konfiguracioni **cfg_reg** registar. **p_offset** i **width** modeluju ulazne vrijednosti za registre IP modula (**p_offset_reg** i **width_reg**) koji čuvaju vrijednost horizontalne pozicije trenutnog klasifikacionog prozora unutar bafera integralne slike, odnosno širinu downsamplovane slike u trenutnoj skali piramide integralnih slika. **bram[]** niz modeluje BRAM memoriju, kao što na isti način **ddr3[]** niz modeluje DDR3 memoriju. Za kraj, **ip_interrupt** i **result** promjenljive modeluju prekid i izlaz kroz koji se rezultat kaskadne klasifikacije vraća nazad procesoru. Unutar glavne funkcije za sintezu, instancionira se objekat klase **VJCore**. Ova klasa sadrži sve potrebne podatke kojima se definiše stanje u kojem se Viola-Jones IP blok nalazi u svakom trenutku.

```
// Stanja IP modula
enum ip_state_t { IDLE, RESET, START };
/* VJCore klasa
// Cuva stanje IP-a izmedju poziva top funkcije.
// Instancira se kao staticka lokalna varijabla unutar top_function tako da
njeni clanovi postaju registri u sintetizovanom RTL-u.
*/
class VJCore{
public:
// IP registri
ap_uint<8> cfg_reg;// bit0=RESET, bit1=START, bit4=DONE
ap_uint<32> p_offset_reg;// x pozicija prozora
ap_uint<32> width_reg;// sz.width tekuce skale
ap_int<32> result_reg;// 1=lice, -i=odbijen na fazi i
bool ip_interrupt;
ip_state_t state;
// Interni registri datapath-a (vrijednosti coskova integralnih slika iz
DDR3)
ap_int<32> p[4];// cetiri coska iz int_img_buffer
ap_int<32> pq[4];// cetiri coska iz sq_int_buffer
VJCore();
};
```

Listing 9. Deklaracija klase VJCore i stanja IP modula u HLS implementaciji

Stanja IP modula su prikazana kodnim listingom iznad, uz deklaraciju VJCore klase, **state** polje predstavlja stanje u kom se IP modul nalazi. Stanje u kome se modul nalazi zavisi od trenutne vrednosti u polju **cfg_reg**, koje modeluje **cfg_reg** registar. Načini na koje top funkcija reaguje na promjene stanja prikazani su na sljedećem kodnom listingu :

```
//reagovanje na promjene stanja
switch(IP.state){
case IDLE:
{
IP.ip_interrupt = false;
break;
}
case RESET:
{
IP.ip_interrupt=false;
IP.result_reg=0;
IP.p_offset_reg=0;
IP.width_reg=0;
reset_l:for(int i = 0; i < 4; i++)
    IP.p[i] = IP.pq[i] = 0;
IP.cfg_reg = ap_uint<8>(0);
break;
}
case START:
{
IP.ip_interrupt = false;
IP.p_offset_reg = p_offset;
IP.width_reg = width;
IP.result_reg = run_cascade_classifier(IP.p_offset_reg, IP.width_reg,
IP.p, IP.pq, bram, ddr3);
IP.cfg_reg.set(IP_DONE_BIT, 1);
IP.cfg_reg.set(IP_START_BIT, 0);
IP.ip_interrupt = true;
break;
}
}
```

Listing 10. Implementacija reakcije sistema na promjene stanja u HLS implementaciji

Moguća su sljedeća stanja:

1. Totalni **reset modula (state = RESET)**, tokom kojeg se sve vrijednosti promjenljivih koje predstavljaju registre IP bloka postavljaju na nulu, kao i flag za interruptove na false, kako bi modul bio inicijalizovan i spreman za dalje korištenje. Takođe, vrši se i reset internih registara koji čuvaju vrijednosti ćoškova trenutnog prozora, za šta je zadužena **reset_l** petlja.
2. **Stanje izvršavanja/počетка izvršavanja kaskadne klasifikacije u okviru modula (state = START)**, sastoji se iz implementacije funkcije **runCascadeClassifier**. Koraci koje modul preduzima u ovom stanju su sljedeći:
 - **inline ažuriranje vrijednosti za piksele u trenutnom prozoru** : IP čita po četiri vrijednosti ćoškova trenutnog klasifikacionog prozora iz bafera integralne i kvadrirane integralne slike koji se nalazi u DDR3, kako bi izračunao normalizacioni faktor za normalizaciju svih slabih klasifikatora u ovom prozoru. DDR3 memorija sadrži integralnu i kvadriranu integralnu sliku za trenutnu skalu piramide slika i trenutni **y_bias** korak.
 - **petlja po fazama kaskade** : Za svaku od 25 faza kaskade čita se broj slabih klasifikatora iz **bram_stages**, a zatim se kreiraju slabi klasifikatori izvršavanjem koda **eval_weak_classifier** pomoćne funkcije za svaki klasifikator. Na kraju svake faze se čita klasifikacioni prag za pripadajuću fazu iz **bram_stages** i akumulirana težinska klasifikaciona suma se poredi sa tim

pragom. Ukoliko je suma manja od praga, prozor se odbija na trenutnoj fazi klasifikacije i, prekida se klasifikacija i vraća se -i vrijednost kao rezultat klasifikacije.

- **pozivi funkcije evalWeakClassifier** : za svaki slabi klasifikator čitaju se **tree_thresh** vrijednosti iz **bram_treeth**, geometrijske vrijednosti Haarovih pravougaonika iz **bram_rect**, težine iz **bram_weights** i vrijednosti bafera integralne slike iz DDR3 memorije. Rezultat ove funkcije je **alpha1** ili **alpha2** vrijednost, zavisno od toga koji je rezultat poređenja akumulisane težinske sume sa pragom klasifikatora skaliranim variance_norm faktorom.

Ako prozor prođe svih 25 faza, funkcija **runCascadeClassifier** vraća 1 (što znači da je detektovano lice), posljedično se izlaz **result_reg** ažurira tako da sadrži rezultat trenutne klasifikacije, u konfiguracionom registru se postavlja DONE bit, a **ip_interrupt** se postavlja na true. Ovakav način postavljanja konfiguracionog bita u istom stanju gdje se odvija klasifikacija je moguć zbog načina rada algoritma, koji obrađuje tačno jedan prozor po jednoj kaskadnoj klasifikaciji, pa ćemo sigurno imati prekid i rezultat klasifikacije nakon obrade jednog. U slučaju drugih algoritama za klasifikaciju i detekciju lica, potreba za završnim (END,STOP) stanjem bi vjerovatno postojala, međutim sama priroda Viola-Jones algoritma nam omogućava da ovo uradimo na prehodno opisani način.

3. **Stanje mirovanja** (state = IDLE) u kome modul ne radi ništa, ip_interrupt signal se postavlja na false kako bi se znalo da se prekid ne dešava, dok nam ostale vrijednosti praktično nisu od interesa, jer se kod za kaskadnu klasifikaciju ne izvršava, niti se dešava upis bita koji signalizuju da je potrebno pokrenuti je.

Za testne ulaze korišćene pri verifikaciji, tijelo funkcije se izvršava 200188 puta – jedan reset ciklus na početku + jednom po prozoru klasifikatora za testnu sliku rezolucije 360×240 piksela, sa faktorom skaliranja 1.2 i shiftStep-om 1. Za procjenu performansi uzima se da IP modul u jednom ciklusu obavlja jedno čitanje iz BRAM-a ili DDR3-a.

Nakon uspješne C simulacije i sinteze dobija se izvještaj o sintetizovanom rješenju za funkciju top_function.

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00 ns	7.157 ns	2.70 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)			
min	max	min	max	min	max	Type	
1	57874	10.000 ns	0.579 ms	2	57875	no	

Detail

Instance

Instance		Module		Latency (cycles)		Latency (absolute)		Interval (cycles)			
				min	max	min	max	min	max	Type	
grp_top_function_Pipeline_reset_l_fu_152	top_function_Pipeline_reset_l	6	6	60.000 ns	60.000 ns	6	6	no			
grp_run_cascade_classifier_fu_172	run_cascade_classifier	29	57873	0.290 us	0.579 ms	29	57873	no			

Loop

N/A

Slika 21. Procjena performansi sintetizovanog rješenja (top_function)

Na osnovu vrijednosti testnih vektora, procjene performansi i uspješnosti C simulacije i sinteze, izveli smo zaključak da je od ukupnog broja izvršavanja, 161355 prozora (80.6%) odbijeno već u fazi 0 (minimalna latencija kaskadne klasifikacije od 29 ciklusa), dok su 94 prozora prošla svih 25 faza (maksimalna latencija od 57873 ciklusa).

Na osnovu samog izvještaja se može zaključiti sljedeće:

1. Minimalna perioda takt signala iznosi 7.157 ns, uz nesigurnost od 2.7 ns. Ciljana perioda takt signala je ostala podrazumijevana, 10 ns. Ovo u praksi znači da je zbog procentualno velike nesigurnosti (koja potiče od same prirode kaskadne klasifikacije), taktni ofset u virtualnoj platformi u okviru IP bloka moguće postaviti isključivo na 10ns, kako bismo izbjegli nepredviđeno ponašanje sistema i zalazak u nedefinisana stanja. Na ovaj način, pokrivamo slučaj maksimalnog vremena trajanja izvršavanja operacija u modulu (minimalna perioda takta + nesigurnost) i obezbjeđujemo neometan i ispravan rad sistema.

2. Broj ciklusa neophodnih da se izvrši jedan prolaz kroz implementiranu funkciju je maksimalno 57875, odnosno, vrijeme potrebno da se jedan prolaz izvrši iznosi maksimalno 0.579 ms. U praksi, zbog prirode kaskadne klasifikacije, veoma mali broj prozora će da preživi klasifikaciju do vremena maksimalnog prolaza funkcije.

3. Kaskadna klasifikacija jednog prozora, kao što smo u prethodnom dijelu ustanovili, traje najmanje 29 ciklusa (0.290 μ s), a najviše 57873 ciklusa (0.579ms).

Vrijeme izvršavanja klasifikacije svih prozora nije moguće precizno izračunati na osnovu HLS izvještaja, jer latencija po prozoru zavisi od broja evaluiranih slabih klasifikatora, što je određeno dinamički tokom izvršavanja. Kao što smo već ustanovili, minimalna latencija iznosi 29 ciklusa (0.290 μ s) za prozore odbijene u fazi 0, a maksimalna 57873 ciklusa (0.579 ms) za prozore koji prođu svih 25 faza. Na osnovu ovih podataka, moguće je analizirati dva krajnja slučaja:

Slučaj 1: Svi prozori prolaze svih 25 faza kaskadne klasifikacije : 200187 prozora * 57873 ciklusa od po 10ns – približno 115,895 sekundi.

Slučaj 2: Nijedan prozor ne prolazi prvu fazu kaskadne klasifikacije (prozori odbijeni u prvoj fazi) : 200187 prozora * 29 ciklusa od po 10ns – približno 0,058 sekundi.

Očigledno, ni jedan ni drugi slučaj se nikada neće dogoditi u realnom sistemu, i nisu pretjerano korisni kao aproksimativne vrijednosti za procjenu trajanja klasifikacije, jer je trajanje klasifikacije izuzetno dinamičko za svaki prozor, pa ih HLS ne može izračunati. S obzirom na to da, po podacima za raspodjelu koje smo predložili, 161355 od 200187 prozora (80.6%) biva odbijeno već u fazi 0, realno ukupno vrijeme izvršavanja je znatno bliže minimalnoj granici. Najbližu moguću procjenu možemo dobiti ubacivanjem podataka o ofsetima dobijenim iz HLS analize u projektovanu virtualnu platformu. Za sliku korištenu za izradu testnih vektora za testiranje HLS implementacije, vrijeme trajanja rada čitave virtualne platforme je 1 059 293 390ns – približno **1.059 sekundi**, s time da će vrijeme trajanja same klasifikacije biti nešto manje jer su u ovo vrijeme uračunati i vremenski troškovi TLM transakcija. Ovo je konzistentno s prethodnim zaključkom da će vrijeme biti mnogo bliže minimalnoj granici klasifikacije.

6.2 Analiza površine

Uređaj se mapira na Zynq-7000 SoC, koji je na Zybo Z7-10 pločici smješten u CLG400 BGA pakovanju, površine 289 mm². Softver se pokreće na ARM Cortex-A9 procesoru, fiksne površine, dok površina koju zauzima sami hardver zavisi od potrošenih hardverskih resursa. Na slici u nastavku data je procjena potrošenih hardverskih resursa potrebnih za implementaciju funkcije koja opisuje IP modul.

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	15	-
FIFO	-	-	-	-	-
Instance	-	18	2382	3569	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	146	-
Register	-	-	290	-	-
Total	0	18	2672	3730	0
Available	120	80	35200	17600	0
Utilization (%)	0	22	7	21	0

Slika 22. Procjena zauzete površine sintetizovanog rješenja (top_function)

Potrebno je ukupno 3584 lookup tabela (LUT) za implementaciju osnovnih aritmetičkih, logičkih i relacionih operacija u implementiranom modulu, dok je 3569 iskorišteno za implementaciju funkcionalnosti same kaskadne klasifikacije. Dodatnih 146 lookup tabela iskorišćeno je za implementaciju multipleksera koji se koriste za kontrolu toka podataka u okviru modula, što znači da je iskorišteno 21% dostupnih LUTova za ovu HLS implementaciju. Ovo znači da je prostor za akceleraciju ogroman u tom aspektu. Nadalje, pošto su BRAM i DDR3 memorije modelovane kao eksterni portovi IP modula, nije potrošena nijedna BRAM ćelija za internu implementaciju - svim statičkim klasifikacionim podacima i baferima integralne slike pristupa se putem vanjskih memorijskih interfejsa. 18 DSP48E1 blokova iskorišćeno je za implementaciju množača koji se koriste u računanju Haarovih karakteristika, varijansnog normalizacionog faktora i adresnoj aritmetici, što nam ostavlja prostor za hardversku akceleraciju i u tom aspektu, jer se koristi svega 22% dostupnih DSP blokova. Ukupno 2672 flipflopova je iskorišćeno za implementaciju registara koji se koriste za skladištenje sadržaja promjenljivih funkcije koja opisuje IP modul, od čega 2382 otpada na samu klasifikaciju, uključujući interne registre koji čuvaju vrijednosti čoškova integralne slike za trenutno obrađivani prozor, kao i registre stanja IP modula, što je takođe pozitivno jer je i u tom aspektu iskorišteno svega 7% dostupnih hardverskih resursa flipflopova.

Pored IP modula, pri mapiranju je potrebno uzeti u obzir i AXI interconnect komponentu, a i potrebno je reći da se zauzima 52484 16-bitnih BRAM lokacija. Svaka BRAM18K ćelija u 16-bit wide režimu može sadržati do 1,024 lokacija ($16,384 \text{ bita} \div 16 \text{ bita} = 1,024$). One se smještaju na 52 BRAM ćelije (51 popunjena i 25% posljednje ćelije) BRAM_18K. AXI interconnect i BRAM kontroler ne doprinose značajno ovim vrijednostima i nisu uzeti u razmatranje u ovoj procjeni.

6.3 Analiza potrošnje i interfejsa

U ovoj procjeni potrošnje, u obzir se uzima samo pomjeranje podataka od i ka DDR3 modulu - najlakši način za to je prebrojavanje TLM transakcija unutar virtuelne platforme. Procesor i DDR3 modul izvrše ukupno 43.743.594 TLM transakcija sa DDR3 modulom u toku čitave klasifikacije. Broj transakcija je veoma veliki, zbog trenutnog nepostojanja DMA modula i posljedičnog individualnog bajt prenosa podatak po podatak iz i u DDR3 modul. Podaci koji su se prenosili su tipa int32_t (vrijednosti integralne i kvadirane integralne slike). U ovom sistemu bezbjednost nije od značaja za rad sistema i samim tim neće biti potrebno koristiti nestandardne interfejse, nego će se u realizaciji sistema koristiti standardni AXI interfejsi, kako bi se IP modul u RTL dijelu projekta lako povezao sa planiranim AXI DMA kontrolerom, BRAM kontrolerom i AXI interconnect komponentom.

Interface

- **Summary**

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	top_function	return value
ap_rst	in	1	ap_ctrl_hs	top_function	return value
ap_start	in	1	ap_ctrl_hs	top_function	return value
ap_done	out	1	ap_ctrl_hs	top_function	return value
ap_idle	out	1	ap_ctrl_hs	top_function	return value
ap_ready	out	1	ap_ctrl_hs	top_function	return value
cfg_val_i	in	8	ap_ovld	cfg_val	pointer
cfg_val_o	out	8	ap_ovld	cfg_val	pointer
cfg_val_o_ap_vld	out	1	ap_ovld	cfg_val	pointer
p_offset	in	32	ap_none	p_offset	scalar
width	in	32	ap_none	width	scalar
bram_address0	out	16	ap_memory	bram	array
bram_ce0	out	1	ap_memory	bram	array
bram_q0	in	16	ap_memory	bram	array
bram_address1	out	16	ap_memory	bram	array
bram_ce1	out	1	ap_memory	bram	array
bram_q1	in	16	ap_memory	bram	array
ddr3_address0	out	15	ap_memory	ddr3	array
ddr3_ce0	out	1	ap_memory	ddr3	array
ddr3_q0	in	32	ap_memory	ddr3	array
ddr3_address1	out	15	ap_memory	ddr3	array
ddr3_ce1	out	1	ap_memory	ddr3	array
ddr3_q1	in	32	ap_memory	ddr3	array
ip_interrupt	out	1	ap_vld	ip_interrupt	pointer
ip_interrupt_ap_vld	out	1	ap_vld	ip_interrupt	pointer
result	out	32	ap_vld	result	pointer
result_ap_vld	out	1	ap_vld	result	pointer

Slika 23. Interfejsi sintetizovanog rješenja (top_function)

Na prethodnoj slici su prikazani korišćeni interfejsi u HLS implementaciji. Ukoliko bi se ova implementacija koristila kao finalna, dovoljno bi bilo učitavanje generisanog RTL modela u formatu sintetizovanog checkpointa u Vivado. Potom, pomoću IP Packager alata u Vivadu moguće je automatski generisati ulazno-izlazne kontrolere za AXI protokole i upakovati sve fajlove koji čine jedno IP jezgro u standardizovani format.

7. Reference

1. Viola, P.; Jones, M. (2001). "Rapid object detection using a boosted cascade of simple features". Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001. Vol. 1. IEEE Comput. Soc. doi:10.1109/cvpr.2001.990517. ISBN 0-7695-1272-0. S2CID 2715202.
2. Grant Martin, Brian Bailey, Andrew Piziali: ESL Design and Verification: A Prescription for Electronic System Level Methodology (Systems on Silicon)
3. Brian Bailey, Grant Martin: ESL Models and their Application: Electronic System Level Design and Verification in Practice
4. Zybo Z7 10: https://digilent.com/reference/_media/reference/programmable-logic/zybo-z7/zybo-z7_rm.pdf
5. Kupusinac, A. : Programski jezik C++, Novi Sad: Fakultet tehničkih nauka, 2020.
6. Vranjković, V. : Projektovanje elektronskih uređaja na sistemskom nivou, Novi Sad: Fakultet tehničkih nauka, 2020.
7. Wikipedia članak o Viola-Jones detekciji objekata : https://en.wikipedia.org/wiki/Viola%E2%80%93Jones_object_detection_framework
8. Valgrind : <https://valgrind.org/info/about.html>
9. Struharik, R. : [literatura sa predmeta Projektovanje složenih digitalnih sistema](#), Novi Sad: Fakultet tehničkih nauka
10. Informacije o AXI interconnect komponenti : [AXI Interconnect](#)